

Every real source file in the repository — 32 files, no lockfile, no generated design canvas, nothing binary. This is a reading copy, not a substitute for the repository: clone it to run it.

github.com/vandakvpllc-dev/the-wire

GENERATED FOR PRINT · SOURCE OF TRUTH IS THE GIT REPO

.env.example	Env template	31
.gitignore	Config	45
AGENTS.md	Markdown	10
CLAUDE.md	Markdown	2
README.md	Markdown	104
app/api/fire/route.ts	TypeScript	139
app/api/start/route.ts	TypeScript	35
app/demo.css	CSS	1264
app/globals.css	CSS	343
app/layout.tsx	TypeScript / React	49
app/page.tsx	TypeScript / React	80
components/Anatomy.tsx	TypeScript / React	134
components/Diagnosis.tsx	TypeScript / React	251
components/Form.tsx	TypeScript / React	193
components/Icons.tsx	TypeScript / React	154
components/Live.tsx	TypeScript / React	306
components/Phone.tsx	TypeScript / React	99
components/Proof.tsx	TypeScript / React	199
components/Rail.tsx	TypeScript / React	65
components/Shell.tsx	TypeScript / React	45
lib/anatomy.ts	TypeScript	59
lib/leads.ts	TypeScript	42
lib/money.ts	TypeScript	28
lib/notify/channels.ts	TypeScript	206
lib/notify/ics.ts	TypeScript	80
lib/notify/index.ts	TypeScript	36
lib/personalize.ts	TypeScript	119

env.example	TypeScript Env template · 31 lines
lib/validate.ts	TypeScript 59
next.config.ts	TypeScript 8
package.json	JSON 24
tsconfig.json	JSON 35
# Copy this file to .env.local and fill in what you have.	
# EVERY line is optional. With none of them set, the demo still runs end to end –	
# it just says honestly on screen that the notification didn't go out.	
# --- The phone buzz -----	
# A real text message. About a penny each, ~\$10 per thousand people.	
# This is the one that makes the moment land: no permission prompt, no app,	
# no home-screen trick, works on every phone.	
TWILIO_ACCOUNT_SID=	
TWILIO_AUTH_TOKEN=	
TWILIO_FROM=+1	
# Email. Free, arrives on their phone in seconds, and carries the calendar	
# invite as a one-tap attachment. Set both or neither.	
RESEND_API_KEY=	
MAIL_FROM=The Wire <hello@yourdomain.com>	
# --- Personalised copy -----	
# Without this the demo uses a hand-written fallback that is already good.	
# With it, every run is written around their own service and price.	
# Costs roughly two cents per HUNDRED people.	
ANTHROPIC_API_KEY=	
# --- Your lead list -----	
# A Google Apps Script web-app URL that appends a row to your sheet.	
# Leads are always written to the server log too, so nothing is ever lost.	
LEADS_WEBHOOK_URL=	

.gitignore

Config · 45 lines

```
# See https://help.github.com/articles/ignoring-files/ for more about ignoring files.

# dependencies
/node_modules
/.pnp
.pnp.*
.yarn/*
!.yarn/patches
!.yarn/plugins
!.yarn/releases
!.yarn/versions

# testing
/coverage

# next.js
/.next/
/out/

# production
/build

# misc
.DS_Store
*.pem

# debug
npm-debug.log*
yarn-debug.log*
yarn-error.log*
.pnpm-debug.log*

# env files (can opt-in for committing if needed)
.env*

# vercel
.vercel

# typescript
*.tsbuildinfo
next-env.d.ts

# Generated by the design tool from the .dc.html sources – 2.5MB of baked-in editor.
design/*-demo.html
```

AGENTS.md

Markdown · 10 lines

```
<!-- BEGIN:nextjs-agent-rules -->
```

```
# This is NOT the Next.js you know
```

This version has breaking changes – APIs, conventions, and file structure may all differ from your training data. Read the relevant guide in ``node_modules/next/dist/docs/`` (resolved from this file's directory; in monorepos the ``next`` package may not be visible from the repo root) before writing any code. Heed deprecation notices.

This block is written and re-added by ``next dev`` – verify at ``node_modules/next/dist/server/lib/generate-agent-files.js``. Removing it from a diff only re-creates the uncommitted change; committing it with your work keeps the tree clean.

```
<!-- END:nextjs-agent-rules -->
```

CLAUDE.md

Markdown · 2 lines

@AGENTS.md

README.md

Markdown · 104 lines

The Wire

A five-screen demo that shows a business owner the difference between **AI automation** and **an AI system** – by running one on their own business while they watch, and putting a green **+\$450.00** on their phone at the end of it.

What a visitor experiences

1. **Five boxes.** Name, email, mobile, what they sell, what they charge. Thirty seconds. The price box is the one that turns everything after it green.
2. **What you have now.** Their assets drawn as disconnected parts, every wire hanging loose. *You are the wire.*
3. **How it actually works.** The five parts every AI system has – **TRIGGER → CAPTURE → DECIDE → ACT → NOTIFY** – then the reveal: ChatGPT is part three, and only part three.
4. **They pull the trigger.** A test payment for their own price. The pulse travels the rail, the log prints every step with real timestamps, and their phone buzzes.
5. **The difference.** Same money, two worlds. Closes on: **\$250** today either way. **\$0** on Wednesday with a smart pen – **\$250** on Wednesday with a system.

The one rule this codebase is built around

The notification is sent before the animation starts. `app/api/fire/route.ts` dispatches to the phone and does *not* await it, then paces the screen at `~850ms` a step for about six seconds. The phone can only ever be early. **Early** reads as magic. **Late** reads as broken.

Two things follow from that:

- The AI copy is generated at `/api/start`, while they read screens 2 and 3 – about forty seconds of runway – and cached on the client. The trigger never waits on a model, and `lib/personalize.ts` has a hand-written fallback that ships fine on its own.
- The timestamps in the log are the true machine times (tenths of a second). The reveal is slower than the truth so a human can read it, and the screen says so rather than pretending otherwise.

Running it

```
```bash
npm install
cp .env.example .env.local # every line is optional
npm run dev # http://localhost:3000
```
```

With no environment variables at all the demo runs end to end and states plainly on screen that the send didn't go out. Nothing fails silently.

What's in the folder

```
```
app/page.tsx the five screens, and the state machine between them
app/api/start/route.ts banks the lead, starts writing their copy
app/api/fire/route.ts the trigger – sends first, then streams the five parts
components/Rail.tsx the five stations. always five across, at every width
components/Live.tsx the money screen: log, phone, and the stream reader
components/Phone.tsx no fake status bar – the real one renders on top
lib/anatomy.ts the five parts, in order. this is the teaching
lib/money.ts their price, formatted, everywhere
lib/notify/ sms, email, and the .ics invite
lib/leads.ts everyone who ran it – this is the list
```
```

What it costs to run

```
Hosting	**$0** – Vercel's free tier carries this
Email buzz	**$0** – free tier covers thousands
```

| AI personalisation | ****~2¢**** per ***hundred*** runs |
 | A real text message | ****~1¢** each** – optional, and the only real cost |

Why nobody gets charged

The payment is a ****test payment**** – a real link and a real flow with no card behind it. The notification that follows is completely real; the charge is the only thing that isn't. That's what lets this go to a thousand strangers without one of them spending a cent.

No OAuth, on purpose

An earlier design signed people in with Google so it could write to their calendar. That puts Google's "this app wants to manage your calendar" consent screen directly in front of the payoff – the worst possible place for a wall. The demo only ever sends ***to*** them, so it needs no permissions at all, and the calendar invite rides along as a one-tap **`.ics`** attachment. OAuth belongs in the paid product, when they're wiring their real business.

It has to fit anyone

A baker doesn't book appointments. A consultant doesn't ship boxes. So nothing in the demo assumes what kind of business it's looking at: the two things the system ***does*** are written per business by **`.lib/personalize.ts`** and carry a **`.kind`** (**`.mail`**, **`.calendar`**, **`.box`**, **`.list`**) that picks the icon on the phone. The calendar invite is only attached when that business actually schedules something. The fallback copy – what runs with no API key – is true of every business on earth.

Verified end to end as a \$450 consultancy and an \$85 cake business.

Known gaps

- Verified at phone width in a desktop browser, not on real hardware. The alternate beat on mobile ("swipe down, it's already waiting" instead of "look down at your hand") is written and detected but unconfirmed on a real device.
- No channel is wired up yet, so the notification currently only lands on screen. That's three keys in **`.env.local`** away.

app/api/fire/route.ts

TypeScript · 139 lines

```

import type { FireEvent, Step } from "@lib/types";
import { moneyIn } from "@lib/money";
import { dispatch } from "@lib/notify";
import { readAnswers, readCopy } from "@lib/validate";

/**
 * The trigger.
 *
 * Order matters more than anything else in this file: the notification is
 * dispatched BEFORE the first byte of the animation. The screen is paced to
 * about six seconds, so the phone can only ever be early. Early reads as magic.
 * Late reads as broken.
 *
 * The timestamps in the log are the real machine times — tenths of a second.
 * The reveal is slower than the truth so a human can read it, and the screen
 * says so rather than pretending otherwise.
 */

const GAP_MS = 850;

/**
 * The stream deliberately takes about six seconds, which is uncomfortably close
 * to the default serverless timeout. Give it room: a cut-off stream means the
 * demo dies halfway through the one moment it exists for.
 */
export const maxDuration = 30;

const enc = new TextEncoder();
const sse = (e: FireEvent) => enc.encode(`data: ${JSON.stringify(e)}\n\n`);
const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));

export async function POST(request: Request) {
  let body: unknown;
  try {
    body = await request.json();
  } catch {
    return Response.json({ error: "bad request" }, { status: 400 });
  }

  const b = body as Record<string, unknown>;
  const answers = readAnswers(b?.answers);
  const copy = readCopy(b?.copy);

  if (!answers || !copy) {
    return Response.json({ error: "incomplete" }, { status: 400 });
  }

  const steps: Step[] = [
    {
      n: 0,
      part: "trigger",
      at: 100,
      text: `received — ${copy.customerName}, ${copy.purchase}`,
      money: moneyIn(answers.price),
    },
    {
      n: 1,
      part: "capture",
      at: 300,
      text: "row 148 written to your sheet — they will be remembered",
    },
    { n: 2, part: "decide", at: 600, text: copy.verdict },
    { n: 3, part: "act", at: 900, text: copy.actNow.text },
    { n: 4, part: "act", at: 900, text: copy.actSecond.text },
    {
      n: 5,
      part: "act",

```



```

    at: 1000,
    text: copy.followUp,
    scheduled: "Wed 9:00 AM",
  },
  { n: 6, part: "notify", at: 1000, text: "pushing to your phone" },
];

// Fires now. Deliberately not awaited – the animation is the delay.
const delivery = dispatch({ answers, copy });

const stream = new ReadableStream<Uint8Array>({
  async start(controller) {
    const startedAt = Date.now();
    let closed = false;

    const send = (e: FireEvent) => {
      if (closed) return;
      try {
        controller.enqueue(sse(e));
      } catch {
        closed = true;
      }
    };

    try {
      for (const step of steps) {
        send({ type: "step", step });
        await sleep(GAP_MS);
      }

      const results = await delivery;
      const landed = results.filter((r) => r.ok).map((r) => r.channel);

      if (landed.length) {
        send({ type: "delivered", channels: landed, at: Date.now() - startedAt });
      } else {
        const why = results.every((r) => r.reason === "not configured")
          ? "no channel configured"
          : (results.find((r) => !r.ok)?.reason ?? "unknown");
        send({ type: "failed", reason: why, at: Date.now() - startedAt });
      }

      send({
        type: "done",
        total: answers.price,
        elapsed: steps[steps.length - 1].at,
      });
    } catch (e) {
      console.error("[the-wire] fire failed:", e);
      send({ type: "failed", reason: "server error", at: Date.now() - startedAt });
    } finally {
      closed = true;
      try {
        controller.close();
      } catch {
        /* already closed by the client navigating away */
      }
    }
  },
});

return new Response(stream, {
  headers: {
    "Content-Type": "text/event-stream; charset=utf-8",
    "Cache-Control": "no-cache, no-transform",
    Connection: "keep-alive",
    // Nginx and some proxies buffer SSE into uselessness without this.
    "X-Accel-Buffering": "no",
  },
});

```

```
});  
}
```

app/api/start/route.ts

TypeScript · 35 lines

```
import { personalize } from "@lib/personalize";
import { recordLead } from "@lib/leads";
import { readAnswers } from "@lib/validate";
import { configuredChannels } from "@lib/notify";

/**
 * Called the instant they submit the five boxes.
 *
 * Two jobs: bank the lead, and start writing their copy while they read the
 * next two screens. By the time they reach the trigger this has long since
 * resolved, so the moment that matters never waits on a model.
 */
export async function POST(request: Request) {
  let body: unknown;
  try {
    body = await request.json();
  } catch {
    return Response.json({ error: "bad request" }, { status: 400 });
  }

  const answers = readAnswers((body as Record<string, unknown>)?.answers);
  if (!answers) {
    return Response.json({ error: "incomplete" }, { status: 400 });
  }

  const source = String((body as Record<string, unknown>)?.source ?? "").slice(0, 60);

  const [copy] = await Promise.all([
    personalize(answers),
    recordLead({ ...answers, at: new Date().toISOString(), source: source || undefined }),
  ]);

  return Response.json({ copy, channels: configuredChannels() });
}
```

app/demo.css

CSS · 1264 lines

```
/* =====  
THE WIRE — screen composition  
  
Five screens, one chassis. The rail and the five-slot strip stay five  
across at every width because the shape of that row is the teaching.  
===== */  
  
/* ----- shell */  
  
.screen {  
  position: relative;  
  z-index: 1;  
  min-height: 100dvh;  
  display: flex;  
  flex-direction: column;  
}  
  
.wrap {  
  width: 100%;  
  max-width: 1240px;  
  margin: 0 auto;  
  padding: 0 clamp(18px, 3.6vw, 56px) clamp(36px, 4vw, 56px);  
  display: flex;  
  flex-direction: column;  
  flex-grow: 1;  
}  
  
.head {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  gap: 16px;  
  flex-wrap: wrap;  
  padding: 22px 0 18px;  
  border-bottom: 1px solid var(--line-soft);  
  margin-bottom: clamp(24px, 3vw, 40px);  
}  
  
.mark {  
  display: flex;  
  align-items: center;  
  gap: 11px;  
  font-family: var(--mono);  
  font-size: 11.5px;  
  font-weight: 500;  
  letter-spacing: 0.28em;  
  color: var(--ink);  
}  
  
.markBadge {  
  display: inline-flex;  
  align-items: center;  
  justify-content: center;  
  width: 28px;  
  height: 28px;  
  border-radius: 8px;  
  background: var(--card-2);  
  border: 1px solid var(--line);  
  box-shadow: var(--edge);  
  flex-shrink: 0;  
}  
  
.headRight {  
  display: flex;  
  align-items: center;  
  gap: 16px;  
  min-width: 0;
```

```
}

.stepLabel {
  font-family: var(--mono);
  font-size: 10px;
  letter-spacing: 0.2em;
  color: var(--fainter);
  text-align: right;
  white-space: nowrap;
}

/* five ticks for five screens – neutral, never green. green is money only. */
.ticks {
  display: flex;
  gap: 5px;
  align-items: center;
}

.tick {
  width: 18px;
  height: 3px;
  border-radius: 99px;
  background: rgba(255, 255, 255, 0.1);
  transition: background 0.4s ease;
}

.tick.on {
  background: var(--ink-3);
}

.livePill {
  display: inline-flex;
  align-items: center;
  gap: 7px;
  font-family: var(--mono);
  font-size: 9.5px;
  font-weight: 500;
  letter-spacing: 0.2em;
  color: var(--money);
  background: var(--money-wash);
  border: 1px solid var(--money-line);
  border-radius: 99px;
  padding: 5px 11px 5px 9px;
  white-space: nowrap;
}

.liveDot {
  width: 6px;
  height: 6px;
  border-radius: 50%;
  background: var(--money);
  box-shadow: 0 0 8px 1px rgba(52, 209, 124, 0.7);
  flex-shrink: 0;
}

.live {
  font-family: var(--mono);
  font-size: 11px;
  letter-spacing: 0.14em;
  color: var(--money);
}

.body {
  flex: 1;
  display: flex;
  flex-direction: column;
  min-width: 0;
}

.body > .stack,
```

```
.body > .formSplit {
  flex: 1;
}

/* ----- typography */

.h1 {
  font-family: var(--serif);
  font-weight: 400;
  font-size: clamp(36px, 5vw, 60px);
  line-height: 1;
  letter-spacing: -0.022em;
  color: var(--ink);
  margin: 0;
  text-wrap: balance;
}

.h2 {
  font-family: var(--serif);
  font-weight: 400;
  font-size: clamp(28px, 3.7vw, 46px);
  line-height: 1.06;
  letter-spacing: -0.02em;
  color: var(--ink);
  margin: 0;
  text-wrap: balance;
}

.h3 {
  font-family: var(--serif);
  font-weight: 400;
  font-size: clamp(22px, 2.1vw, 28px);
  line-height: 1.14;
  letter-spacing: -0.015em;
  color: var(--ink);
  margin: 0;
}

.lede {
  font-size: clamp(15.5px, 1.15vw, 17px);
  color: var(--muted);
  line-height: 1.62;
  max-width: 62ch;
  text-wrap: pretty;
  margin: 0;
}

.eyebrow {
  font-family: var(--mono);
  font-size: 10px;
  font-weight: 500;
  letter-spacing: 0.22em;
  color: var(--faint);
}

.t-body {
  font-size: 15px;
  color: var(--ink-3);
  line-height: 1.6;
}

.t-sm {
  font-size: 13.5px;
  color: var(--muted);
  line-height: 1.55;
}

.t-xs {
  font-size: 12px;
  color: var(--faint);
}
```

```

    line-height: 1.5;
  }

.t-lab {
  font-family: var(--mono);
  font-size: 9.5px;
  font-weight: 500;
  letter-spacing: 0.2em;
  color: var(--fainter);
}

.stack {
  display: flex;
  flex-direction: column;
  min-width: 0;
}

/* a hairline that fades out – reads as drawn, not as a divider component */
.rule {
  height: 1px;
  background: linear-gradient(90deg, var(--line-hard), transparent);
}

.ruleMoney {
  height: 1px;
  background: linear-gradient(90deg, var(--money-line-2), transparent);
}

/* ----- form */

.formGrid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 14px 18px;
}

.field {
  display: flex;
  flex-direction: column;
  gap: 8px;
  min-width: 0;
}

.field > label {
  font-family: var(--mono);
  font-size: 10px;
  font-weight: 500;
  letter-spacing: 0.17em;
  color: var(--faint);
}

.field input {
  background: rgba(255, 255, 255, 0.032);
  border: 1px solid var(--line);
  border-radius: var(--r);
  padding: 14px 15px;
  font-size: 16px; /* 16px keeps iOS from zooming the page on focus */
  color: var(--ink);
  box-shadow: var(--edge);
  width: 100%;
  outline: none;
  transition: border-color 0.16s ease, box-shadow 0.16s ease, background 0.16s ease;
}

.field input::placeholder {
  color: var(--fainter);
}

.field input:hover {
  border-color: rgba(255, 255, 255, 0.13);
}

```

```

}

.field input:focus {
  border-color: rgba(255, 255, 255, 0.3);
  background: rgba(255, 255, 255, 0.055);
  box-shadow: var(--edge), 0 0 4px rgba(255, 255, 255, 0.05);
}

/* the price is the number the entire demo runs on. it gets typeset like one. */
.field.moneyField > label {
  color: var(--money);
}

.field.moneyField input {
  border-color: var(--money-line);
  background: var(--money-wash-2);
  font-family: var(--mono);
  font-size: 19px;
  font-weight: 500;
  letter-spacing: -0.01em;
  color: var(--money);
  padding: 11px 15px;
  box-shadow: var(--edge), 0 0 26px -14px rgba(52, 209, 124, 0.8);
}

.field.moneyField input::placeholder {
  color: rgba(52, 209, 124, 0.34);
}

.field.moneyField input:hover {
  border-color: rgba(52, 209, 124, 0.48);
}

.field.moneyField input:focus {
  border-color: var(--money);
  background: var(--money-wash);
  box-shadow: var(--edge), 0 0 4px rgba(52, 209, 124, 0.14),
    0 0 34px -12px rgba(52, 209, 124, 0.9);
}

.field.moneyField .withSign {
  position: relative;
}

.field.moneyField .withSign span {
  position: absolute;
  left: 15px;
  top: 50%;
  transform: translateY(-50%);
  font-family: var(--mono);
  font-size: 19px;
  color: rgba(52, 209, 124, 0.62);
  font-weight: 500;
  pointer-events: none;
}

.field.moneyField .withSign input {
  padding-left: 28px;
}

.field .err {
  font-size: 12px;
  color: var(--alert);
}

/* ----- buttons */

.go {
  position: relative;
  display: inline-flex;

```



```

    align-items: center;
    gap: 12px;
    background: linear-gradient(180deg, var(--money-bright), var(--money));
    color: var(--money-ink);
    border-radius: var(--r);
    padding: 0 24px;
    font-size: 15px;
    font-weight: 600;
    letter-spacing: -0.005em;
    min-height: 52px;
    box-shadow:
      inset 0 1px 0 rgba(255, 255, 255, 0.4),
      0 1px 2px rgba(0, 0, 0, 0.4),
      0 16px 42px -16px rgba(52, 209, 124, 0.75);
    transition: transform 0.16s ease, box-shadow 0.2s ease, filter 0.16s ease;
    white-space: nowrap;
  }

  .go svg {
    transition: transform 0.22s cubic-bezier(0.22, 1, 0.36, 1);
  }

  .go:hover {
    filter: brightness(1.07);
    transform: translateY(-1px);
    box-shadow:
      inset 0 1px 0 rgba(255, 255, 255, 0.45),
      0 1px 2px rgba(0, 0, 0, 0.4),
      0 22px 54px -16px rgba(52, 209, 124, 0.9);
  }

  .go:hover svg {
    transform: translateX(3px);
  }

  .go:active {
    transform: translateY(0);
    filter: brightness(0.97);
  }

  .go[disabled] {
    background: rgba(255, 255, 255, 0.055);
    color: var(--fainter);
    cursor: default;
    filter: none;
    transform: none;
    box-shadow: var(--edge);
  }

  .ghost {
    display: inline-flex;
    align-items: center;
    gap: 10px;
    border: 1px solid var(--line-hard);
    border-radius: var(--r);
    padding: 0 20px;
    font-size: 14px;
    font-weight: 500;
    color: var(--ink-3);
    min-height: 48px;
    transition: border-color 0.16s ease, color 0.16s ease, background 0.16s ease;
  }

  .ghost:hover {
    border-color: rgba(255, 255, 255, 0.3);
    color: var(--ink);
    background: rgba(255, 255, 255, 0.03);
  }

  /* ----- cards */

```

```

.card {
  position: relative;
  background: linear-gradient(180deg, rgba(255, 255, 255, 0.032), rgba(255, 255, 255, 0.008)),
    var(--card);
  border: 1px solid var(--line);
  border-radius: var(--r-lg);
  box-shadow: var(--edge), var(--shadow-sm);
}

/* a surface that has been pressed into the page — logs, panels, reveals */
.cardSunk {
  position: relative;
  background: var(--sunk);
  border: 1px solid var(--line-soft);
  border-radius: var(--r-lg);
  box-shadow: inset 0 1px 2px rgba(0, 0, 0, 0.6);
}

/* disconnected. no light on it, nothing holding it to anything. */
.cardDead {
  position: relative;
  background: rgba(255, 255, 255, 0.012);
  border: 1px dashed var(--line-hard);
  border-radius: var(--r-lg);
  transition: border-color 0.2s ease;
}

/* money moved through here */
.cardMoney {
  position: relative;
  background: linear-gradient(180deg, rgba(52, 209, 124, 0.085), rgba(52, 209, 124, 0.015)),
    var(--card);
  border: 1px solid var(--money-line);
  border-radius: var(--r-lg);
  box-shadow: inset 0 1px 0 rgba(107, 234, 168, 0.16),
    0 24px 60px -28px rgba(52, 209, 124, 0.6);
}

/* ----- rail */

.rail {
  position: relative;
  padding: 4px 0 2px;

  /* the fill has to land exactly on a station node, and the nodes are centred
    in grid columns — so the gap is part of the arithmetic, not decoration */
  --rail-gap: clamp(6px, 1.6vw, 26px);
  --rail-inset: 8%;
  --rail-col: calc((100% - 4 * var(--rail-gap)) / 5);
}

.railLine,
.railFill {
  position: absolute;
  top: 23px;
  height: 2px;
  border-radius: 2px;
}

/* an unconnected run of wire is drawn, not filled */
.railLine {
  left: var(--rail-inset);
  right: var(--rail-inset);
  background-image: linear-gradient(
    90deg,
    var(--line-hard) 0 6px,
    transparent 6px 12px
  );
  background-size: 12px 2px;
}

```

```

background-repeat: repeat-x;
}

.railFill {
  left: var(--rail-inset);
  width: calc(
    var(--i, 0) * (var(--rail-col) + var(--rail-gap)) + var(--rail-col) / 2 -
    var(--rail-inset)
  );
  background: linear-gradient(90deg, var(--money-deep), var(--money) 55%, var(--money-bright));
  box-shadow: 0 0 18px -2px rgba(52, 209, 124, 0.65);
  transition: width 0.85s cubic-bezier(0.22, 1, 0.36, 1), opacity 0.3s ease;
}

/* the leading edge of the pulse */
.railFill[data-complete]::after {
  display: none;
}

.railFill::after {
  content: "";
  position: absolute;
  right: -3px;
  top: 50%;
  transform: translateY(-50%);
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: var(--money-bright);
  box-shadow: 0 0 14px 3px rgba(107, 234, 168, 0.75);
}

.railGrid {
  display: grid;
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: var(--rail-gap);
  position: relative;
}

.station {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 13px;
  text-align: center;
  min-width: 0;
}

.dot {
  width: 14px;
  height: 14px;
  border-radius: 50%;
  margin-top: 17px;
  background: var(--paper);
  border: 2px solid var(--line-hard);
  box-sizing: border-box;
  flex-shrink: 0;
  transition: background 0.35s ease, border-color 0.35s ease, box-shadow 0.35s ease;
}

.dot.on {
  background: var(--money);
  border-color: var(--money);
  box-shadow: 0 0 12px 1px rgba(52, 209, 124, 0.45);
}

.dot.now {
  background: var(--money-bright);
  border-color: var(--money-bright);
  animation: nodePulse 1.5s ease-out infinite;
}

```

```
}

.stationName {
  font-family: var(--mono);
  font-size: clamp(9px, 1vw, 11.5px);
  font-weight: 500;
  letter-spacing: 0.14em;
  color: var(--fainter);
  transition: color 0.35s ease;
}

.stationName.on {
  color: var(--ink);
}

.stationSub {
  font-family: var(--mono);
  font-size: clamp(8.5px, 0.85vw, 10.5px);
  color: var(--faint);
}

/* ----- console */

.console {
  display: flex;
  flex-direction: column;
  overflow: hidden;
}

.consoleBar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  padding: 13px 20px;
  border-bottom: 1px solid var(--line-soft);
  background: rgba(255, 255, 255, 0.018);
  flex-wrap: wrap;
}

.consoleBody {
  display: flex;
  flex-direction: column;
  gap: 2px;
  padding: 14px 12px 18px;
  min-height: 268px;
}

.consoleFoot {
  margin-top: auto;
  border-top: 1px solid var(--line-soft);
  padding: 15px 20px;
  display: flex;
  align-items: flex-start;
  gap: 12px;
  background: rgba(255, 255, 255, 0.014);
}

.logRow {
  display: grid;
  grid-template-columns: 62px 68px minmax(0, 1fr);
  gap: 14px;
  align-items: baseline;
  font-family: var(--mono);
  padding: 8px 10px 8px 12px;
  border-radius: 7px;
  border-left: 2px solid transparent;
  position: relative;
}
```

```
.logRow:nth-child(odd) {
  background: rgba(255, 255, 255, 0.016);
}

.logRow.hot {
  border-left-color: var(--money);
  background: var(--money-wash-2);
}

.logTime {
  font-size: 11px;
  color: var(--fainter);
  font-variant-numeric: tabular-nums;
}

.logPart {
  font-size: 10px;
  font-weight: 500;
  color: var(--money);
  letter-spacing: 0.13em;
}

.logText {
  font-size: 12.5px;
  color: var(--ink-2);
  line-height: 1.5;
  overflow-wrap: anywhere;
}

/* ----- phone */

.phoneCol {
  display: flex;
  flex-direction: column;
}

/* the light the device sits in */
.phoneStage {
  position: relative;
  display: flex;
  justify-content: center;
  padding: 8px 0 4px;
}

.phoneStage::before {
  content: "";
  position: absolute;
  left: 50%;
  top: 12%;
  width: 78%;
  height: 76%;
  transform: translateX(-50%);
  border-radius: 50%;
  background: radial-gradient(closest-side, rgba(52, 209, 124, 0.16), transparent 72%);
  filter: blur(26px);
  pointer-events: none;
}

.phone {
  position: relative;
  width: min(296px, 100%);
  background: linear-gradient(180deg, #16191A 0%, #0D100F 42%, #0A0C0B 100%);
  border: 1px solid rgba(255, 255, 255, 0.11);
  border-radius: 34px;
  padding: 34px 13px 15px;
  min-height: 452px;
  display: flex;
  flex-direction: column;
  gap: 9px;
  box-shadow:
```

```

    inset 0 1px 0 rgba(255, 255, 255, 0.14),
    inset 0 0 0 1px rgba(0, 0, 0, 0.6),
    var(--shadow-lg);
}

/* real notifications are frosted slabs floating on the wallpaper */
.notif {
  background: rgba(255, 255, 255, 0.075);
  -webkit-backdrop-filter: blur(14px);
  backdrop-filter: blur(14px);
  border: 1px solid rgba(255, 255, 255, 0.07);
  border-radius: 17px;
  padding: 12px 14px;
  display: flex;
  flex-direction: column;
  gap: 6px;
  box-shadow: 0 6px 18px -8px rgba(0, 0, 0, 0.8);
}

.notif.money {
  background: linear-gradient(180deg, rgba(52, 209, 124, 0.2), rgba(52, 209, 124, 0.09));
  border-color: var(--money-line-2);
  box-shadow: 0 0 0 1px rgba(52, 209, 124, 0.14),
    0 14px 34px -12px rgba(52, 209, 124, 0.55);
}

.notif.waiting {
  background: rgba(255, 255, 255, 0.022);
  border: 1px dashed var(--line-hard);
  backdrop-filter: none;
  -webkit-backdrop-filter: none;
  box-shadow: none;
}

.notifHead {
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 11px;
  color: var(--ink-3);
  letter-spacing: 0.01em;
}

.notifWhen {
  font-family: var(--mono);
  font-size: 9.5px;
  color: var(--faint);
  margin-left: auto;
}

.notifAmount {
  font-family: var(--mono);
  font-size: 28px;
  color: var(--money-bright);
  font-weight: 500;
  letter-spacing: -0.02em;
  line-height: 1.05;
  text-shadow: 0 0 22px rgba(52, 209, 124, 0.5);
  font-variant-numeric: tabular-nums;
}

.notifBody {
  font-size: 12.5px;
  color: var(--ink-2);
  line-height: 1.45;
}

.notifFoot {
  margin-top: auto;
  display: flex;

```

```

    flex-direction: column;
    align-items: center;
    gap: 4px;
    padding: 15px 6px 6px;
    text-align: center;
}

.appIcon {
    width: 19px;
    height: 19px;
    border-radius: 6px;
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0;
    background: rgba(255, 255, 255, 0.16);
    color: var(--ink);
}

.appIcon.moneyIcon {
    background: linear-gradient(180deg, var(--money-bright), var(--money));
    color: var(--money-ink);
    box-shadow: 0 0 12px -2px rgba(52, 209, 124, 0.9);
}

/* ----- grids */

.two {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: clamp(16px, 2.2vw, 28px);
}

.assetGrid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(228px, 1fr));
    gap: 12px;
}

/* Wide cards that fill the second row – but only where two columns fit. */
.spanTwo {
    grid-column: span 2;
}

.revealGrid {
    display: grid;
    grid-template-columns: minmax(280px, 480px) minmax(0, 1fr);
    gap: clamp(22px, 3.4vw, 48px);
    align-items: center;
}

/* The you/you/the-AI/you/you strip. Same rule as the rail: it stays five
   across at every width, because the shape IS the argument. */
.fiveTight {
    display: grid;
    grid-template-columns: repeat(5, minmax(0, 1fr));
    gap: clamp(4px, 1vw, 10px);
}

.slot {
    border: 1px dashed var(--line-hard);
    border-radius: 8px;
    padding: clamp(12px, 1.5vw, 17px) 4px;
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 7px;
    text-align: center;
    min-width: 0;
}

```

```
.slot.ai {
  border: 1px solid var(--money-line-2);
  background: linear-gradient(180deg, rgba(52, 209, 124, 0.14), rgba(52, 209, 124, 0.03));
  box-shadow: 0 0 0 1px rgba(52, 209, 124, 0.08),
    0 16px 36px -18px rgba(52, 209, 124, 0.75);
}

.slotName {
  font-family: var(--mono);
  font-size: clamp(6.5px, 1.5vw, 9px);
  font-weight: 500;
  letter-spacing: 0.06em;
  color: var(--fainter);
}

.slot.ai .slotName {
  color: var(--money);
}

.slotWho {
  font-size: clamp(11px, 2vw, 14px);
  font-weight: 600;
  color: var(--faint);
  letter-spacing: -0.01em;
}

.slot.ai .slotWho {
  color: var(--ink);
}

.five {
  display: grid;
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: clamp(10px, 1.4vw, 16px);
}

/* one of the five parts, on the anatomy screen */
.partCard {
  padding: 18px 17px 17px;
  display: flex;
  flex-direction: column;
  gap: 13px;
  min-width: 0;
}

.partTop {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 8px;
}

.partNum {
  font-family: var(--mono);
  font-size: 22px;
  font-weight: 400;
  line-height: 1;
  letter-spacing: -0.03em;
  color: var(--fainter);
  font-variant-numeric: tabular-nums;
}

.cardMoney .partNum {
  color: var(--money);
}

.partIcon {
  display: inline-flex;
  align-items: center;
```



```
    justify-content: center;
    width: 32px;
    height: 32px;
    border-radius: 9px;
    border: 1px solid var(--line);
    background: rgba(255, 255, 255, 0.03);
    box-shadow: var(--edge);
    flex-shrink: 0;
}

.cardMoney .partIcon {
    border-color: var(--money-line);
    background: rgba(52, 209, 124, 0.1);
}

.partName {
    font-family: var(--mono);
    font-size: 11.5px;
    font-weight: 500;
    letter-spacing: 0.18em;
    color: var(--ink);
}

.partText {
    font-size: 13px;
    color: var(--muted);
    line-height: 1.58;
    flex-grow: 1;
}

.partTool {
    font-family: var(--mono);
    font-size: 10px;
    letter-spacing: 0.04em;
    color: var(--fainter);
}

.liveSplit {
    display: grid;
    grid-template-columns: minmax(0, 1fr) 312px;
    gap: clamp(20px, 3vw, 44px);
    align-items: stretch;
}

.formSplit {
    display: grid;
    grid-template-columns: minmax(0, 1fr) 340px;
    gap: clamp(28px, 4vw, 64px);
    align-items: start;
}

.stats {
    display: flex;
    gap: clamp(18px, 2.6vw, 32px);
    flex-wrap: wrap;
}

.stat {
    display: flex;
    flex-direction: column;
    gap: 5px;
    min-width: 0;
}

.statNum {
    font-family: var(--mono);
    font-size: clamp(24px, 2.5vw, 32px);
    font-weight: 400;
    line-height: 1;
    letter-spacing: -0.03em;
}
```

```

    color: var(--ink);
    font-variant-numeric: tabular-nums;
  }

  .statNum.xl {
    font-size: clamp(46px, 6vw, 74px);
    letter-spacing: -0.045em;
    font-feature-settings: "zero" 1;
  }

  .statLabel {
    font-family: var(--mono);
    font-size: 9px;
    font-weight: 500;
    letter-spacing: 0.2em;
    color: var(--fainter);
  }

  .statMoney {
    border-left: 2px solid var(--money);
    padding-left: 16px;
  }

  .statMoney .statNum {
    color: var(--money);
    text-shadow: 0 0 24px rgba(52, 209, 124, 0.4);
  }

  .statMoney .statLabel {
    color: var(--money);
  }

  /* ----- proof: two worlds */

  .worldDead {
    padding: 24px 24px 22px;
    display: flex;
    flex-direction: column;
    gap: 18px;
  }

  .worldLive {
    padding: 24px 24px 22px;
    display: flex;
    flex-direction: column;
    gap: 18px;
  }

  .stepRow {
    display: flex;
    align-items: center;
    gap: 13px;
    border: 1px dashed var(--line-hard);
    border-radius: 8px;
    padding: 11px 14px;
    min-width: 0;
  }

  .stepRow.isLive {
    border: 1px solid var(--money-line);
    background: var(--money-wash-2);
  }

  .stepRow.isAi {
    border-style: solid;
    border-color: var(--line);
    background: rgba(255, 255, 255, 0.03);
  }

  .stepWho {

```

```

    font-family: var(--mono);
    font-size: 10px;
    font-weight: 500;
    letter-spacing: 0.12em;
    width: 26px;
    flex-shrink: 0;
    color: var(--fainter);
}

.stepRow.isLive .stepWho {
    color: var(--money);
}

.stepRow.isAi .stepWho {
    color: var(--ink-3);
}

.stepText {
    font-size: 12.5px;
    line-height: 1.45;
    color: var(--faint);
}

.stepRow.isLive .stepText,
.stepRow.isAi .stepText {
    color: var(--ink-2);
}

.ledger {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(212px, 1fr));
    gap: clamp(18px, 2.4vw, 30px);
    padding: 24px 26px;
    align-items: center;
}

.ledgerCell {
    display: flex;
    flex-direction: column;
    gap: 7px;
    border-left: 1px solid var(--line);
    padding-left: 20px;
    min-width: 0;
}

.ledgerCell.money {
    border-left: 2px solid var(--money);
}

.ledgerNum {
    font-family: var(--mono);
    font-size: clamp(26px, 2.6vw, 34px);
    line-height: 1;
    letter-spacing: -0.035em;
    color: var(--ink);
    font-variant-numeric: tabular-nums;
}

.ledgerNum.dead {
    color: var(--faint);
}

.ledgerNum.win {
    color: var(--money);
    font-weight: 500;
    text-shadow: 0 0 26px rgba(52, 209, 124, 0.45);
}

/* ----- shared blocks */

```

```

/* headline on the left, the one button on the right */
.closer {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  gap: clamp(24px, 4vw, 48px);
  flex-wrap: wrap;
}

.noteBox {
  background: var(--loss-wash);
  border: 1px solid var(--loss-line);
  border-left: 2px solid var(--line-hard);
  border-radius: var(--r);
  padding: 18px 20px;
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.wiringNote {
  display: flex;
  flex-direction: column;
  gap: 8px;
  border-left: 2px solid var(--money);
  padding-left: 20px;
  min-width: 240px;
}

/* ----- breakpoints */

@media (max-width: 1120px) {
  .five {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
}

@media (max-width: 1040px) {
  .liveSplit,
  .formSplit {
    grid-template-columns: minmax(0, 1fr);
  }

  .phoneCol {
    align-items: center;
  }

  /* The rail is the teaching device – it stays five-across at every width. */
  .railGrid {
    grid-template-columns: repeat(5, minmax(0, 1fr));
  }
}

@media (max-width: 860px) {
  .five {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .revealGrid {
    grid-template-columns: minmax(0, 1fr);
  }
}

@media (max-width: 720px) {
  .two,
  .formGrid,
  .five,
  .assetGrid,
  .revealGrid {
    grid-template-columns: minmax(0, 1fr);
  }
}

```

```

}

/* One column means nothing can span two without pushing the page sideways. */
.spanTwo {
  grid-column: auto;
}

.logRow {
  grid-template-columns: 58px minmax(0, 1fr);
  row-gap: 3px;
}

.logPart {
  grid-column: 2;
}

.logText {
  grid-column: 2;
}

.rail {
  --rail-inset: 10%;
}

.stationSub {
  display: none;
}

.stepLabel {
  display: none;
}

.head {
  padding: 16px 0 14px;
}

.ledger {
  padding: 20px 20px;
}

.ledgerCell {
  border-left: none;
  border-top: 1px solid var(--line);
  padding-left: 0;
  padding-top: 14px;
}

.ledgerCell.money {
  border-left: none;
  border-top: 2px solid var(--money);
}

.worldDead,
.worldLive {
  padding: 20px 18px;
}

.consoleBody {
  padding: 12px 8px 16px;
  min-height: 220px;
}

.wiringNote {
  min-width: 0;
}
}

@media (max-width: 400px) {
  .stats {
    gap: 16px;
  }
}

```

```
}  
  
.statMoney {  
  padding-left: 12px;  
}  
}
```

app/globals.css

CSS · 343 lines

```

/* =====
THE WIRE – design tokens

One rule governs the whole palette: money is the only colour. Everything
that is dead, disconnected or waiting is drawn in neutral graphite; the
instant money moves, it turns green. Nothing else in this product is
allowed to be chromatic, so green never has to compete for attention.

The ground is a near-black graphite rather than paper, because the product
is an instrument – a rail, a log, a phone – and instruments read on black.
Green on black is a live wire. Green on cream is a spreadsheet.
===== */

:root {
  /* ground */
  --void: #060807;
  --paper: #0B0D0C;
  --card: #101312;
  --card-2: #161A19;
  --sunk: #090B0A;

  /* lines – all alpha, so they sit correctly on any surface above */
  --line: rgba(255, 255, 255, 0.078);
  --line-soft: rgba(255, 255, 255, 0.045);
  --line-hard: rgba(255, 255, 255, 0.155);
  --dashed: rgba(255, 255, 255, 0.19);

  /* ink */
  --ink: #F3F6F4;
  --ink-2: #DBE0DD;
  --ink-3: #B7BEBB;
  --muted: #9BA29F;
  --faint: #848B88;
  --fainter: #6C7371;

  /* money – the only accent. grey when dead, green when money moves */
  --money: #34D17C;
  --money-bright: #6BEAA8;
  --money-deep: #0F8F52;
  --money-ink: #04130A;
  --money-wash: rgba(52, 209, 124, 0.1);
  --money-wash-2: rgba(52, 209, 124, 0.05);
  --money-line: rgba(52, 209, 124, 0.32);
  --money-line-2: rgba(52, 209, 124, 0.55);

  /* loss is not a colour. dead things are grey – that is the argument. */
  --loss: #9CA3A0;
  --loss-wash: rgba(255, 255, 255, 0.022);
  --loss-line: rgba(255, 255, 255, 0.1);

  /* the single functional exception: an inline form error must read as a
  problem, so it gets a low-chroma clay that cannot be mistaken for money */
  --alert: #D6A38C;

  --shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.45);
  --shadow-md: 0 2px 6px rgba(0, 0, 0, 0.4), 0 24px 60px -22px rgba(0, 0, 0, 0.85);
  --shadow-lg: 0 4px 14px rgba(0, 0, 0, 0.45), 0 48px 110px -32px rgba(0, 0, 0, 0.95);
  --shadow-money: 0 0 1px var(--money-line), 0 20px 50px -20px rgba(52, 209, 124, 0.45);

  /* the light-catching top edge that makes a surface read as raised */
  --edge: inset 0 1px 0 rgba(255, 255, 255, 0.055);
  --edge-strong: inset 0 1px 0 rgba(255, 255, 255, 0.1);

  --r-sm: 6px;
  --r: 10px;
  --r-lg: 14px;
  --r-xl: 20px;

```

```

--serif: var(--font-serif), "Iowan Old Style", Georgia, serif;
--sans: var(--font-sans), -apple-system, BlinkMacSystemFont, "Helvetica Neue", sans-serif;
--mono: var(--font-mono), ui-monospace, SFMono-Regular, Menlo, monospace;

color-scheme: dark;
}

*,
*::before,
*::after {
  box-sizing: border-box;
}

html,
body {
  margin: 0;
  padding: 0;
}

html {
  background: var(--paper);
}

body {
  min-height: 100dvh;
  color: var(--ink);
  font-family: var(--sans);
  font-size: 16px;
  line-height: 1.5;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-rendering: optimizeLegibility;
  font-feature-settings: "kern" 1, "liga" 1;

  /* a drafting grid, barely there – the wiring diagram the demo is about */
  background-color: var(--paper);
  background-image:
    linear-gradient(rgba(255, 255, 255, 0.016) 1px, transparent 1px),
    linear-gradient(90deg, rgba(255, 255, 255, 0.016) 1px, transparent 1px);
  background-size: 36px 36px, 36px 36px;
  background-position: center top;
}

/* one soft light source from above, and a floor that falls away */
body::before {
  content: "";
  position: fixed;
  inset: 0;
  pointer-events: none;
  z-index: 0;
  background:
    radial-gradient(110% 62% at 50% -12%, rgba(255, 255, 255, 0.05), transparent 62%),
    radial-gradient(90% 55% at 50% 112%, rgba(0, 0, 0, 0.55), transparent 60%);
}

/* film grain – 3%, enough to kill the plastic flatness of a flat black */
body::after {
  content: "";
  position: fixed;
  inset: 0;
  pointer-events: none;
  z-index: 0;
  opacity: 0.032;
  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='180' height='180'%3E%3Cfilter
id='n'%3E%3CfeTurbulence type='fractalNoise' baseFrequency='0.82' numOctaves='3' stitchTiles='stitch'/%3E%3C/filter%3E%3Crect
width='180' height='180' filter='url(%23n)'/%3E%3C/svg%3E");
}

img {

```



```
    max-width: 100%;
}

[hidden] {
    display: none !important;
}

button {
    font: inherit;
    color: inherit;
    border: none;
    background: none;
    cursor: pointer;
}

input {
    font: inherit;
    color: inherit;
}

a {
    color: var(--money);
    text-decoration-thickness: 1px;
    text-underline-offset: 3px;
}

a:hover {
    color: var(--money-bright);
}

::selection {
    background: rgba(52, 209, 124, 0.3);
    color: #fff;
}

:focus-visible {
    outline: 2px solid var(--money);
    outline-offset: 3px;
    border-radius: 3px;
}

/* dark autofill, or Chrome paints a 1998 yellow rectangle over the design */
input:-webkit-autofill,
input:-webkit-autofill:hover,
input:-webkit-autofill:focus {
    -webkit-text-fill-color: var(--ink);
    -webkit-box-shadow: 0 0 0 1000px #14201A inset;
    caret-color: var(--ink);
    transition: background-color 9999s ease-out;
}

::-webkit-scrollbar {
    width: 10px;
    height: 10px;
}

::-webkit-scrollbar-track {
    background: transparent;
}

::-webkit-scrollbar-thumb {
    background: rgba(255, 255, 255, 0.1);
    border-radius: 99px;
    border: 3px solid transparent;
    background-clip: content-box;
}

::-webkit-scrollbar-thumb:hover {
    background: rgba(255, 255, 255, 0.18);
    background-clip: content-box;
}
```

```

}

.mono {
  font-family: var(--mono);
  font-feature-settings: "kern" 1;
}

.serif {
  font-family: var(--serif);
  font-weight: 400;
}

.italic {
  font-style: italic;
}

/* the demo runs long-form; never let a wide row scroll the page */
.scroll-x {
  overflow-x: auto;
}

/* ----- motion */

@keyframes livePulse {
  0%,
  100% {
    opacity: 0.35;
  }
  50% {
    opacity: 1;
  }
}

@keyframes nodePulse {
  0%,
  100% {
    box-shadow: 0 0 0 rgba(52, 209, 124, 0.42), 0 0 14px 2px rgba(52, 209, 124, 0.5);
  }
  70% {
    box-shadow: 0 0 0 9px rgba(52, 209, 124, 0), 0 0 18px 4px rgba(52, 209, 124, 0.28);
  }
}

@keyframes caret {
  0%,
  49% {
    opacity: 1;
  }
  50%,
  100% {
    opacity: 0;
  }
}

@keyframes riseIn {
  from {
    opacity: 0;
    transform: translateY(7px);
  }
  to {
    opacity: 1;
    transform: none;
  }
}

@keyframes screenIn {
  from {
    opacity: 0;
    transform: translateY(12px);
  }
}

```

```

    to {
      opacity: 1;
      transform: none;
    }
  }

@keyframes moneyLand {
  0% {
    opacity: 0;
    transform: translateY(-16px) scale(0.96);
    filter: blur(4px);
  }
  60% {
    transform: translateY(2px) scale(1.008);
    filter: blur(0);
  }
  100% {
    opacity: 1;
    transform: none;
    filter: blur(0);
  }
}

@keyframes markFlicker {
  0%,
  100% {
    opacity: 1;
  }
  50% {
    opacity: 0.55;
  }
}

.pulse {
  animation: livePulse 1.5s ease-in-out infinite;
}

.caret {
  animation: caret 1s step-end infinite;
}

.rise {
  animation: riseIn 0.44s cubic-bezier(0.22, 1, 0.36, 1) both;
}

.land {
  animation: moneyLand 0.62s cubic-bezier(0.22, 1, 0.36, 1) both;
}

.enter {
  animation: screenIn 0.55s cubic-bezier(0.22, 1, 0.36, 1) both;
}

@media (prefers-reduced-motion: reduce) {
  .pulse,
  .caret,
  .rise,
  .land,
  .enter,
  .dot.now {
    animation: none !important;
  }

  * {
    transition-duration: 0.01ms !important;
  }
}

```

app/layout.tsx

TypeScript / React · 49 lines

```
import type { Metadata, Viewport } from "next";
import { Archivo, IBM_Plex_Mono, Instrument_Serif } from "next/font/google";
import "./globals.css";
import "./demo.css";

const archivo = Archivo({
  subsets: ["latin"],
  weight: ["400", "500", "600", "700"],
  variable: "--font-sans",
  display: "swap",
});

const serif = Instrument_Serif({
  subsets: ["latin"],
  weight: "400",
  style: ["normal", "italic"],
  variable: "--font-serif",
  display: "swap",
});

const mono = IBM_Plex_Mono({
  subsets: ["latin"],
  weight: ["400", "500"],
  variable: "--font-mono",
  display: "swap",
});

export const metadata: Metadata = {
  title: "The Wire",
  description:
    "You're not using AI. You're working for it. Make it work for you – that's what it's for. Ninety seconds, on your own business.",
};

export const viewport: Viewport = {
  width: "device-width",
  initialScale: 1,
  themeColor: "#0B0D0C",
};

export default function RootLayout({
  children,
}: Readonly<{ children: React.ReactNode }>) {
  return (
    <html lang="en" className={` ${archivo.variable} ${serif.variable} ${mono.variable}`}>
      <body>{children}</body>
    </html>
  );
}
```

app/page.tsx

TypeScript / React · 80 lines

```

"use client";

import { useCallback, useRef, useState } from "react";
import { Form } from "@components/Form";
import { Diagnosis } from "@components/Diagnosis";
import { Anatomy } from "@components/Anatomy";
import { Live } from "@components/Live";
import { Proof } from "@components/Proof";
import { fallback } from "@lib/personalize";
import type { Answers, Personalized } from "@lib/types";

type Stage = "form" | "diagnosis" | "anatomy" | "live" | "proof";

export default function Page() {
  const [stage, setStage] = useState<Stage>("form");
  const [answers, setAnswers] = useState<Answers | null>(null);
  const [copy, setCopy] = useState<Personalized | null>(null);

  /**
   * Their copy is written while they read the diagnosis and the anatomy –
   * roughly forty seconds of runway – so the trigger never waits on a model.
   */
  const pending = useRef<Promise<Personalized> | null>(null);

  const start = useCallback((a: Answers) => {
    setAnswers(a);
    setStage("diagnosis");

    pending.current = (async () => {
      try {
        const res = await fetch("/api/start", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            answers: a,
            source: new URLSearchParams(window.location.search).get("from") ?? "",
          }),
        });
        if (!res.ok) throw new Error(`start ${res.status}`);
        const data = (await res.json()) as { copy: Personalized };
        return data.copy;
      } catch (e) {
        console.error("[the-wire] start failed, using fallback", e);
        return fallback(a);
      }
    })();

    pending.current.then(setCopy);
  }, []);

  const toLive = useCallback(async () => {
    if (!answers) return;
    // Practically always already resolved; this is the seatbelt.
    const c = copy ?? (await (pending.current ?? Promise.resolve(fallback(answers))));
    setCopy(c);
    setStage("live");
  }, [answers, copy]);

  const restart = useCallback(() => {
    setAnswers(null);
    setCopy(null);
    pending.current = null;
    setStage("form");
  }, []);

  if (stage === "form" || !answers) return <Form onSubmit={start} />;

```

```
if (stage === "diagnosis") {  
  return <Diagnosis a={answers} onNext={() => setStage("anatomy")} />;  
}  
  
if (stage === "anatomy") return <Anatomy a={answers} onNext={toLive} />;  
  
if (stage === "live" && copy) {  
  return <Live a={answers} copy={copy} onNext={() => setStage("proof")} />;  
}  
  
return <Proof a={answers} onRestart={restart} />;  
}
```

components/Anatomy.tsx

TypeScript / React · 134 lines

```

"use client";

import { Shell } from "../Shell";
import { Rail } from "../Rail";
import { Arrow, Bell, Bolt, Brain, Database, Send } from "../Icons";
import { PARTS } from "@lib/anatomy";
import { money, moneyIn } from "@lib/money";
import type { Answers, PartId } from "@lib/types";

const ICONS = [Bolt, Database, Brain, Send, Bell];
const ALL = new Set<PartId>(PARTS.map((p) => p.id));

export function Anatomy({ a, onNext }: { a: Answers; onNext: () => void }) {
  const price = money(a.price);

  return (
    <Shell step={3} label="HOW IT ACTUALLY WORKS">
      <div className="stack" style={{ gap: "clamp(26px, 3.2vw, 40px)" }}>
        <div className="closer">
          <div className="stack" style={{ gap: 16, maxWidth: 720 }}>
            <h1 className="h1">
              Every system is <em className="italic">five</em> parts.
            </h1>
            <p className="lede">
              Not five hundred. Five. Once you can name them, you can look at any business
              – yours, your sister’s, your competitor’s – and put your finger
              on the exact part that’s missing.
            </p>
          </div>
          <div className="wiringNote">
            <span className="eyebrow" style={{ color: "var(--money)" }}>
              WIRING FOR YOU NOW
            </span>
            <span style={{ fontSize: 15, fontWeight: 600, lineHeight: 1.4 }}>
              Your {price} link → your phone
            </span>
            <span
              className="mono"
              style={{ fontSize: 11, color: "var(--money)", letterSpacing: "0.02em" }}
            >
              connected · 6.2 seconds
            </span>
          </div>
        </div>
      </div>

      <Rail done={ALL} />

      <div className="five">
        {PARTS.map((p, i) => {
          const Icon = ICONS[i];
          const last = i === PARTS.length - 1;
          return (
            <div
              key={p.id}
              className={`partCard ${last ? "cardMoney" : "card"} `}
            >
              <div className="partTop">
                <span className="partNum">0{i + 1}</span>
                <span className="partIcon">
                  <Icon color={last ? "var(--money-bright)" : "var(--faint)"} />
                </span>
              </div>
              <span className="partName">{p.label}</span>
              <span className="partText">{p.plain({ price, sells: a.sells })}</span>
              <div className={last ? "ruleMoney" : "rule"} />
              {last ? (
                <span

```

```

        className="mono"
        style={{
          fontSize: 17,
          color: "var(--money)",
          fontWeight: 500,
          letterSpacing: "-0.02em",
          textShadow: "0 0 22px rgba(52,209,124,.45)",
        }}
      >
        {moneyIn(a.price)}
      </span>
    ) : (
      <span className="partTool">{p.tool}</span>
    )}
  </div>
);
}}
</div>

<div
  className="cardSunk revealGrid"
  style={{ padding: "clamp(24px, 3vw, 40px) clamp(22px, 3vw, 40px)" }}
>
  <div className="stack" style={{ gap: 14 }}>
    <h2 className="h2">
      ChatGPT is part three.
      <br />
      You're the <em className="italic">other four.</em>
    </h2>
    <p className="lede" style={{ fontSize: 15 }}>
      That's the entire misunderstanding. It's brilliant at deciding,
      and it owns none of the wires on either side – so you fetch and carry for it,
      forever, for free, at eleven o'clock at night. That is working for the
      AI. It's meant to be the other way round.
    </p>
  </div>

  <div className="fiveTight">
    {PARTS.map((p, i) => {
      const isAi = i === 2;
      return (
        <div className={isAi ? "slot ai" : "slot"} key={p.id}>
          <span className="slotName">{p.label}</span>
          <span className="slotWho">{isAi ? "the AI" : "you"}</span>
        </div>
      );
    })}
  </div>
</div>

<div
  className="closer"
  style={{ alignItems: "center", marginTop: "auto", paddingTop: 8 }}
>
  <p className="lede" style={{ maxWidth: 620 }}>
    Enough theory. Buy something from yourself and watch all five parts run.
  </p>
  <button className="go" onClick={onNext}>
    <span>Pull the trigger</span>
    <Arrow />
  </button>
</div>
</div>
</Shell>
);
}

```


components/Diagnosis.tsx

TypeScript / React · 251 lines

```

"use client";

import { Shell } from "../Shell";
import { Arrow, CalendarIcon, CardIcon, ChatIcon, GlobeIcon, Person, Warn } from "../Icons";
import { money } from "@lib/money";
import type { Answers } from "@lib/types";

/** A port with nothing plugged into it. The whole diagnosis in one glyph. */
function DeadPort() {
  return (
    <svg width="46" height="12" viewBox="0 0 46 12" fill="none" aria-hidden="true">
      <path
        d="M0 6h34"
        stroke="var(--dashed)"
        strokeWidth="1.5"
        strokeDasharray="4 5"
        strokeLinecap="round"
      />
      <circle
        cx="40"
        cy="6"
        r="3.5"
        fill="none"
        stroke="var(--dashed)"
        strokeWidth="1.5"
      />
    </svg>
  );
}

function Asset({
  icon,
  kind,
  title,
  under,
  loss,
  badge,
}: {
  icon: React.ReactNode;
  kind: string;
  title: string;
  under: string;
  loss?: boolean;
  badge?: string;
}) {
  return (
    <div
      className="cardDead"
      style={{
        padding: "17px 18px 18px",
        display: "flex",
        flexDirection: "column",
        gap: 7,
      }}
    >
      <div style={{ display: "flex", alignItems: "center", gap: 10, marginBottom: 4 }}>
        {icon}
        <span className="t-lab">{kind}</span>
        {badge && (
          <span
            className="mono"
            style={{
              fontSize: 11,
              color: "var(--money)",
              marginLeft: "auto",
              fontWeight: 500,
            }}
          >

```

```

    >
    {badge}
  </span>
)}
<span style={{ marginLeft: badge ? 8 : "auto", display: "flex" }}>
  <DeadPort />
</span>
</div>
<span style={{ fontSize: 15, fontWeight: 600, color: "var(--ink-2)" }}>{title}</span>
<span style={{ fontSize: 12.5, color: loss ? "var(--loss)" : "var(--faint)" }}>
  {under}
</span>
</div>
);
}

export function Diagnosis({ a, onNext }: { a: Answers; onNext: () => void }) {
  const price = money(a.price);

  return (
    <Shell step={2} label="WHAT YOU HAVE NOW">
      <div className="stack" style={{ gap: "clamp(22px, 2.8vw, 34px)" }}>
        <div className="stack" style={{ gap: 18 }}>
          <span className="eyebrow">
            {a.firstName.toUpperCase()}, HERE'S WHAT YOU ALREADY OWN
          </span>

          <div className="assetGrid">
            <Asset
              icon={<CardIcon />}
              kind="PAYMENT LINK"
              title="Money arrives"
              under="Then absolutely nothing."
              badge={`+${price}`}
              loss
            />
            <Asset
              icon={<CalendarIcon />}
              kind="SIGN-UP FORM"
              title="Someone signs up"
              under="It tells no one. Not even you."
            />
            <Asset
              icon={<GlobeIcon />}
              kind="WEBSITE"
              title="People visit"
              under="Nothing ever leaves it."
            />
            <Asset
              icon={<ChatIcon />}
              kind="YOUR DMS"
              title="They ask"
              under="You answer. At 11pm. Yourself."
            />

            {/* the one solid, lit object on this screen – because it is the
              only thing on it that actually connects anything */}
            <div
              className="card spanTwo"
              style={{
                padding: "18px 20px 20px",
                display: "flex",
                flexDirection: "column",
                gap: 8,
                borderColor: "var(--line-hard)",
                boxShadow: "var(--edge-strong), var(--shadow-md)",
              }}
            >
              <div style={{ display: "flex", alignItems: "center", gap: 10, marginBottom: 2 }}>
                <Person />

```

```

    <span
      className="mono"
      style={{
        fontSize: 9.5,
        fontWeight: 500,
        letterSpacing: "0.2em",
        color: "var(--ink-3)",
      }}
    >
      THE ONLY WIRE IN THIS PICTURE
    </span>
  </div>
  <span
    className="serif"
    style={{ fontSize: 30, lineHeight: 1.05, letterSpacing: "-0.02em" }}
  >
    {a.firstName}
  </span>
  <span className="t-sm">
    Copying, pasting and remembering, by hand, every single time – and only
    while awake.
  </span>
</div>

<div
  className="cardDead spanTwo"
  style={{
    padding: "17px 18px 18px",
    display: "flex",
    flexDirection: "column",
    gap: 7,
  }}
>
  <span className="t-lab" style={{ marginBottom: 4 }}>
    YOUR PHONE
  </span>
  <span style={{ fontSize: 15, fontWeight: 600, color: "var(--faint)" }}>
    Silent
  </span>
  <span style={{ fontSize: 12.5, color: "var(--fainter)" }}>
    Nothing here can reach it.
  </span>
</div>
</div>
</div>

<div className="two" style={{ alignItems: "stretch" }}>
  <div
    style={{
      display: "flex",
      gap: "clamp(28px, 5vw, 56px)",
      alignItems: "flex-start",
      alignSelf: "center",
    }}
  >
    <div className="stat">
      <span className="statNum xl">5</span>
      <span className="t-sm">things you own</span>
    </div>
    <div className="stat">
      <span className="statNum xl" style={{ color: "var(--fainter)" }}>
        0
      </span>
      <span className="t-sm">wires between them</span>
    </div>
  </div>
  </div>

  <div className="noteBox">
    <div style={{ display: "flex", alignItems: "center", gap: 9 }}>
      <Warn />
    </div>
  </div>

```

```

    <span
      className="mono"
      style={{
        fontSize: 9.5,
        fontWeight: 500,
        letterSpacing: "0.2em",
        color: "var(--loss)",
      }}
    >
      WHERE THE MONEY GOES
    </span>
  </div>
  <span className="t-body">
    A <b style={{ color: "var(--ink)", fontWeight: 600 }}>{price}</b> payment
    landing in your account is the{" "}
    <b style={{ color: "var(--ink)", fontWeight: 600 }}>beginning</b> of a
    relationship worth several times that. Right now it's the end of one —
    because the moment the money arrives, nothing else happens.
  </span>
</div>
</div>

<div className="closer" style={{ marginTop: "auto", paddingTop: 12 }}>
  <div className="stack" style={{ gap: 16, maxWidth: 800 }}>
    <h2 className="h1">
      Right now, <em className="italic">you</em> work for it.
    </h2>
    <p className="lede">
      You don't have a system. You have things that each work alone, and a
      person carrying messages between them by hand. That person is you.
      You're the wire — and the wire is the part that gets tired, forgets,
      and sleeps.
    </p>
  </div>
  <button className="go" onClick={onNext}>
    <span>Show me the wire</span>
    <Arrow />
  </button>
</div>
</div>
</Shell>
);
}

```

components/Form.tsx

TypeScript / React · 193 lines

```

"use client";

import { useState } from "react";
import { Shell } from "../Shell";
import { Phone } from "../Phone";
import { Arrow } from "../Icons";
import { parsePrice } from "@lib/money";
import type { Answers } from "@lib/types";

const FIELDS = [
  { key: "firstName", label: "FIRST NAME", type: "text", placeholder: "Denise", auto: "given-name" },
  { key: "email", label: "EMAIL", type: "email", placeholder: "you@yourbusiness.com", auto: "email" },
  { key: "mobile", label: "MOBILE – SO IT CAN REACH YOU", type: "tel", placeholder: "(214) 555-0148", auto: "tel" },
] as const;

export function Form({ onSubmit }: { onSubmit: (a: Answers) => void }) {
  const [v, setV] = useState({
    firstName: "",
    email: "",
    mobile: "",
    sells: "",
    price: "",
  });
  const [touched, setTouched] = useState(false);

  const price = parsePrice(v.price);
  const emailOk = /^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$/.test(v.email.trim());
  const ready =
    v.firstName.trim().length > 0 && emailOk && v.sells.trim().length > 0 && price !== null;

  const set = (k: keyof typeof v) => (e: React.ChangeEvent<HTMLInputElement>) =>
    setV((p) => ({ ...p, [k]: e.target.value }));

  const submit = (e: React.FormEvent) => {
    e.preventDefault();
    setTouched(true);
    if (!ready || price === null) return;
    onSubmit({
      firstName: v.firstName.trim(),
      email: v.email.trim(),
      mobile: v.mobile.trim(),
      sells: v.sells.trim(),
      price,
    });
  };

  return (
    <Shell step={1} label="THIRTY SECONDS">
      <div className="formSplit">
        <form className="stack" style={{ gap: 34 }} onSubmit={submit} noValidate>
          <div className="stack" style={{ gap: 18 }}>
            <h1 className="h1">
              Five boxes. Then watch
            <br />
              your own business <em className="italic">run itself.</em>
            </h1>
            <p className="lede">
              You’ve been working for the AI. This is what it looks like when it
              works for you. Fill this in and the next four screens are built out of your
              answers – what you sell, your price, your phone. No card, nothing to install,
              nothing to cancel later.
            </p>
          </div>

          <div className="stack" style={{ gap: 14 }}>
            <div className="formGrid">
              {FIELDS.slice(0, 2).map((f) => (

```

```

<div className="field" key={f.key}>
  <label htmlFor={f.key}>{f.label}</label>
  <input
    id={f.key}
    name={f.key}
    type={f.type}
    autoComplete={f.auto}
    placeholder={f.placeholder}
    value={v[f.key]}
    onChange={set(f.key)}
  />
  {touched && f.key === "email" && v.email && !emailOk && (
    <span className="err">That address looks incomplete.</span>
  )}
</div>
  )})
</div>

<div className="formGrid">
  <div className="field">
    <label htmlFor="mobile">{FIELDS[2].label}</label>
    <input
      id="mobile"
      name="mobile"
      type="tel"
      autoComplete="tel"
      placeholder={FIELDS[2].placeholder}
      value={v.mobile}
      onChange={set("mobile")}
    />
  </div>

  <div className="field moneyField">
    <label htmlFor="price">WHAT DO YOU CHARGE FOR IT?</label>
    <div className="withSign">
      <span aria-hidden="true">$</span>
      <input
        id="price"
        name="price"
        type="text"
        inputMode="numeric"
        placeholder="250"
        value={v.price}
        onChange={set("price")}
      />
    </div>
  </div>
</div>

<div className="field">
  <label htmlFor="sells">WHAT DO YOU SELL?</label>
  <input
    id="sells"
    name="sells"
    type="text"
    placeholder="A coaching call, a custom cake, a website audit..."
    value={v.sells}
    onChange={set("sells")}
  />
</div>
</div>

<div style={{ display: "flex", alignItems: "center", gap: 22, flexWrap: "wrap" }}>
  <button className="go" type="submit" disabled={!ready}>
    <span>Build my system</span>
    <Arrow />
  </button>
  <span className="t-xs" style={{ color: "var(--fainter)" }}>
    Takes about 90 seconds.
  </span>
</div>

```

```

        Nothing is charged and nothing is sold to you.
    </span>
</div>
</form>

<div className="stack phoneCol" style={{ gap: 20 }}>
  <span
    className="eyebrow"
    style={{ textAlign: "center", letterSpacing: "0.13em", fontSize: 9.5 }}
  >
    IN ABOUT NINETY SECONDS, THIS IS YOUR PHONE
  </span>
  <div className="phoneStage">
    <Phone
      price={price ?? 250}
      customerName="Maria Ruiz"
      purchase={v.sells.trim().toLowerCase() || "the thing you sell"}
      actNow={{
        text: "receipt and welcome note sent from your own address",
        kind: "mail",
      }}
      actSecond={{
        text: "added to your customer list with everything they bought",
        kind: "list",
      }}
      followUp="The next offer sends itself in three days."
      paid
      didNow
      didSecond
      queued={false}
      footer={
        <>
          <span
            className="mono pulse"
            style={{
              fontSize: 9.5,
              letterSpacing: "0.18em",
              color: "var(--money)",
              fontWeight: 500,
            }}
          >
            ● NOT A MOCKUP
          </span>
          <span style={{ fontSize: 11, color: "var(--faint)" }}>
            Your real phone. Your real number.
          </span>
        </>
      }
    </div>
  </div>
</div>
</Shell>
);
}

```

components/Icons.tsx

TypeScript / React · 154 lines

```

/** Stroke icons on a 24 grid. Never emoji – they don't recolor and don't scale. */

interface P {
  size?: number;
  color?: string;
  className?: string;
}

const base = (size: number, color: string) => ({
  width: size,
  height: size,
  viewBox: "0 0 24 24",
  fill: "none",
  stroke: color,
  strokeWidth: 1.6,
  strokeLinecap: "round" as const,
  strokeLinejoin: "round" as const,
});

export const WireMark = ({ size = 17, color = "var(--money)" }: P) => (
  <svg {...base(size, color)} strokeWidth={1.8}>
    <path d="M2.5 12h5.5" />
    <path d="M16 12h5.5" />
    <circle cx="12" cy="12" r="3.4" />
    <circle cx="12" cy="12" r="1.1" fill={color} stroke="none" />
  </svg>
);

export const Bolt = ({ size = 19, color = "var(--money)" }: P) => (
  <svg {...base(size, color)}>
    <path d="M13 2.5 4 14h6.5L11 21.5 20 10h-6.5z" />
  </svg>
);

export const Database = ({ size = 19, color = "var(--money)" }: P) => (
  <svg {...base(size, color)}>
    <ellipse cx="12" cy="6" rx="8" ry="3" />
    <path d="M4 6v12c0 1.7 3.6 3 8 3s8-1.3 8-3V6" />
    <path d="M4 12c0 1.7 3.6 3 8 3s8-1.3 8-3" />
  </svg>
);

export const Brain = ({ size = 19, color = "var(--money)" }: P) => (
  <svg {...base(size, color)}>
    <path d="M12 3a4 4 0 0 0 4 4v0a3.5 3.5 0 0 0 1.5 6.6A3.5 3.5 0 0 0 9 20.5a3 3 0 0 0 3-1.6 3 3 0 0 0 3 1.6 3.5 3.5 0 0 0 2.5-6.9A3.5 3.5 0 0 0 16 7v0a4 4 0 0 0 4-4z" />
  </svg>
);

export const Send = ({ size = 19, color = "var(--money)" }: P) => (
  <svg {...base(size, color)}>
    <path d="M3 11.5 21 4l-7.5 17.5-2.2-7.8z" />
    <path d="M11.3 13.7 21 4" />
  </svg>
);

export const Bell = ({ size = 19, color = "var(--money)" }: P) => (
  <svg {...base(size, color)}>
    <path d="M18 8.5a6 6 0 0 0 12 0c0 6-2.5 7.5-2.5 7.5h17S18 14.5 18 8.5z" />
    <path d="M10.3 20a2 2 0 0 0 3.4 0" />
  </svg>
);

export const Arrow = ({ size = 16, color = "currentColor" }: P) => (
  <svg {...base(size, color)} strokeWidth={2}>
    <path d="M4 12h15M13 6l6 6 6 6" />
  </svg>
);

```



```

);

export const Clock = ({ size = 15, color = "var(--money)" }: P) => (
  <svg {...base(size, color)}>
    <circle cx="12" cy="12" r="9" />
    <path d="M12 7v5.5l3.5 2" />
  </svg>
);

export const Warn = ({ size = 15, color = "var(--loss)" }: P) => (
  <svg {...base(size, color)} strokeWidth={1.8}>
    <path d="M12 8.5v5" />
    <circle cx="12" cy="17" r="0.6" fill={color} stroke="none" />
    <path d="M10.3 3.9 2.6 17.5A1.9 1.9 0 0 0 4.3 20.4h15.4a1.9 1.9 0 0 0 1.7-2.9L13.7 3.9a1.9 1.9 0 0 0 0-3.4 0z" />
  </svg>
);

export const Person = ({ size = 15, color = "var(--ink)" }: P) => (
  <svg {...base(size, color)} strokeWidth={1.7}>
    <circle cx="12" cy="8" r="4" />
    <path d="M4.5 21c1.2-4 4-6 7.5-6s6.3 2 7.5 6" />
  </svg>
);

/* --- app glyphs, drawn white inside a filled rounded square --- */

export const DollarGlyph = ({ size = 11 }: P) => (
  <svg {...base(size, "currentColor")} strokeWidth={2.6}>
    <path d="M12 4v16M8.5 7.5h6.5a3 3 0 0 1 0 6H9a3 3 0 0 0 0 6h6.5" />
  </svg>
);

export const CalendarGlyph = ({ size = 10 }: P) => (
  <svg {...base(size, "currentColor")} strokeWidth={2.6}>
    <rect x="4" y="5" width="16" height="15" rx="2" />
    <path d="M4 10h16" />
  </svg>
);

export const MailGlyph = ({ size = 10 }: P) => (
  <svg {...base(size, "currentColor")} strokeWidth={2.6}>
    <rect x="3" y="5.5" width="18" height="13" rx="2" />
    <path d="M3.5 7l8.5 6 8.5-6" />
  </svg>
);

export const BoxGlyph = ({ size = 10 }: P) => (
  <svg {...base(size, "currentColor")} strokeWidth={2.6}>
    <path d="M3 8.5 12 4l9 4.5v7L12 20l-9-4.5z" />
    <path d="M3 8.5 12 13l9-4.5M12 13v7" />
  </svg>
);

export const ListGlyph = ({ size = 10 }: P) => (
  <svg {...base(size, "currentColor")} strokeWidth={2.6}>
    <path d="M9 6h11M9 12h11M9 18h11M4.5 6h.01M4.5 12h.01M4.5 18h.01" />
  </svg>
);

/* --- asset glyphs for the map --- */

export const CardIcon = ({ size = 17, color = "var(--faint)" }: P) => (
  <svg {...base(size, color)} strokeWidth={1.5}>
    <rect x="2" y="6" width="20" height="13" rx="2" />
    <path d="M2 10.5h20" />
  </svg>
);

export const CalendarIcon = ({ size = 17, color = "var(--faint)" }: P) => (
  <svg {...base(size, color)} strokeWidth={1.5}>

```

```
    <rect x="3" y="5" width="18" height="16" rx="2" />
    <path d="M3 10h18M8 3v4M16 3v4" />
  </svg>
);

export const GlobeIcon = ({ size = 17, color = "var(--faint)" }: P) => (
  <svg {...base(size, color)} strokeWidth={1.5}>
    <circle cx="12" cy="12" r="9" />
    <path d="M3 12h18M12 3c2.5 2.6 2.5 15.4 0 18M12 3c-2.5 2.6-2.5 15.4 0 18" />
  </svg>
);

export const ChatIcon = ({ size = 17, color = "var(--faint)" }: P) => (
  <svg {...base(size, color)} strokeWidth={1.5}>
    <path d="M21 11.5a8.4 8.4 0 0 1-9 8.4 9 9 0 0 1-4.2-1L3 20.5l1.7-4.6A8.4 8.4 0 0 1 12 3.1a8.4 8.4 0 0 1 9 8.4z" />
  </svg>
);
```

components/Live.tsx

TypeScript / React · 306 lines

```

"use client";

import { useEffect, useRef, useState } from "react";
import { Shell } from "../Shell";
import { Rail } from "../Rail";
import { Phone } from "../Phone";
import { Arrow, Clock } from "../Icons";
import { moneyIn } from "@lib/money";
import type { Answers, FireEvent, PartId, Personalized, Step } from "@lib/types";

const fmt = (ms: number) => `${(ms / 1000).toFixed(1)}s`;

export function Live({
  a,
  copy,
  onNext,
}: {
  a: Answers;
  copy: Personalized;
  onNext: () => void;
}) {
  const [steps, setSteps] = useState<Step[]>([]);
  const [done, setDone] = useState(false);
  const [delivery, setDelivery] = useState<
    { ok: true; channels: string[] } | { ok: false; reason: string } | null
  >(null);
  const [onPhone, setOnPhone] = useState(false);
  const started = useRef(false);
  const alive = useRef(true);

  useEffect(() => {
    setOnPhone(window.matchMedia("(max-width: 720px), (pointer: coarse)").matches);
  }, []);

  useEffect(() => {
    // React double-invokes effects in dev. The trigger must fire exactly once,
    // and – critically – the first invocation's cleanup must NOT kill it, or
    // the one moment this whole demo exists for silently never happens. So the
    // request is never aborted; only the state updates are gated. The stream
    // closes itself after about six seconds either way.
    alive.current = true;
    if (started.current) {
      return () => {
        alive.current = false;
      };
    }
    started.current = true;

    (async () => {
      try {
        const res = await fetch("/api/fire", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ answers: a, copy }),
        });

        if (!res.ok || !res.body) throw new Error(`fire ${res.status}`);

        const reader = res.body.getReader();
        const dec = new TextDecoder();
        let buf = "";

        for (;;) {
          const { done: end, value } = await reader.read();
          if (end) break;
          if (!alive.current) continue;
          buf += dec.decode(value, { stream: true });
        }
      }
    })();
  });
}

```

```

const frames = buf.split("\n\n");
buf = frames.pop() ?? "";

for (const frame of frames) {
  const line = frame.trim();
  if (!line.startsWith("data:")) continue;

  let e: FireEvent;
  try {
    e = JSON.parse(line.slice(5).trim());
  } catch {
    continue;
  }

  if (e.type === "step") {
    setSteps((prev) =>
      prev.some((s) => s.n === e.step.n) ? prev : [...prev, e.step],
    );
  } else if (e.type === "delivered") {
    setDelivery({ ok: true, channels: e.channels });
  } else if (e.type === "failed") {
    setDelivery({ ok: false, reason: e.reason });
  } else if (e.type === "done") {
    setDone(true);
  }
}
} catch (err) {
  console.error("[the-wire] stream failed", err);
  if (alive.current) {
    setDelivery({ ok: false, reason: "connection lost" });
    setDone(true);
  }
}
})();

return () => {
  alive.current = false;
};
}, [a, copy]);

const reached = new Set<PartId>(steps.map((s) => s.part));
const current = steps.length ? steps[steps.length - 1].part : null;
const finished = new Set<PartId>([...reached].filter((p) => p !== current || done));
const elapsed = steps.length ? steps[steps.length - 1].at : 0;

const has = (n: number) => steps.some((s) => s.n === n);

return (
  <Shell step={4} label="YOU PULL THE TRIGGER" live={!done}>
    <div className="stack" style={{ gap: "clamp(24px, 3vw, 34px)" }}>
      <div className="closer">
        <div className="stack" style={{ gap: 16, maxWidth: 690 }}>
          <h1 className="h1">You just paid yourself {moneyIn(a.price).slice(1)}.</h1>
          <p className="lede">
            That was a test payment, so no card was charged. Everything after it is
            completely real.{""}
            {onPhone
              ? "Watch this screen – then swipe down. It is already waiting in your notifications."
              : "Watch the left side, then look down at your hand."}
          </p>
        </div>

        <div className="stats">
          <div className="stat">
            <span className="statNum">{fmt(elapsed)}</span>
            <span className="statLabel">ELAPSED</span>
          </div>
          <div className="stat">

```

```

    <span className="statNum">0</span>
    <span className="statLabel">CLICKS BY YOU</span>
  </div>
  <div className="stat statMoney">
    <span className="statNum">{moneyIn(a.price)}</span>
    <span className="statLabel">COLLECTED</span>
  </div>
</div>
</div>

<Rail
  done={finished}
  now={done ? null : current}
  subs=[
    finished.has("trigger") ? "done · 0.1s" : null,
    finished.has("capture") ? "done · 0.3s" : null,
    finished.has("decide") ? "done · 0.6s" : null,
    reached.has("act") ? (finished.has("act") ? "done · 1.0s" : "running") : null,
    reached.has("notify") ? "your phone" : "next · your phone",
  ]
/>

<div className="liveSplit">
  <div className="card console">
    <div className="consoleBar">
      <span
        className="mono"
        style={{
          fontSize: 10,
          fontWeight: 500,
          letterSpacing: "0.2em",
          color: "var(--ink-3)",
        }}
      >
        WHAT IT IS DOING, RIGHT NOW
      </span>
      <span
        className="mono"
        style={{ fontSize: 10, letterSpacing: "0.1em", color: "var(--fainter)" }}
      >
        nothing hidden
      </span>
    </div>

    <div className="consoleBody">
      {steps.map((s, i) => (
        <div
          className={`logRow rise${i === steps.length - 1 && !done ? " hot" : ""}`}
          key={s.n}
        >
          <span className="logTime">+{(s.at / 1000).toFixed(1)}s</span>
          <span className="logPart">{s.part.toUpperCase()}</span>
          <span className="logText">
            {s.money && (
              <b style={{ color: "var(--money)", fontWeight: 500 }}>{s.money}</b>
            )}
            {s.text}
            {s.scheduled && (
              <span style={{ color: "var(--fainter)" }}>- {s.scheduled}</span>
            )}
            {s.part === "notify" && !done && (
              <span className="caret" style={{ color: "var(--money)" }}>
                |
              </span>
            )}
          </span>
        </div>
      ))}

      {steps.length === 0 && (

```

```

    <span
      className="mono pulse"
      style={{ fontSize: 12.5, color: "var(--faint)", padding: "6px 12px" }}
    >
      waiting for the payment...
    </span>
  )}
</div>

<div className="consoleFoot">
  <span style={{ paddingTop: 2, display: "flex" }}>
    <Clock />
  </span>
  <span style={{ fontSize: 12.5, color: "var(--muted)", lineHeight: 1.5 }}>
    Those timestamps are the real machine times – the whole thing took under a
    second. It is replayed slowly here so you can read it. One of those steps
    is parked for{" "}
    <b style={{ color: "var(--ink-2)", fontWeight: 600 }}>Wednesday morning</b>,
    and it will happen whether or not you ever open this page again.
  </span>
</div>
</div>

<div className="phoneCol">
  <div className="phoneStage">
    <Phone
      price={a.price}
      customerName={copy.customerName}
      purchase={copy.purchase}
      actNow={copy.actNow}
      actSecond={copy.actSecond}
      followUp={copy.followUp}
      paid={has(0)}
      didNow={has(3)}
      didSecond={has(4)}
      queued={has(5)}
      footer={
        delivery?.ok ? (
          <>
            <span
              className="mono"
              style={{
                fontSize: 9.5,
                fontWeight: 500,
                letterSpacing: "0.18em",
                color: "var(--money)",
              }}
            >
              ● SENT TO YOUR {delivery.channels.includes("sms") ? "PHONE" : "INBOX"}
            </span>
            <span style={{ fontSize: 10.5, color: "var(--faint)" }}>
              {delivery.channels.includes("sms") ? a.mobile || a.email : a.email}
            </span>
          </>
        ) : delivery && !delivery.ok ? (
          <span style={{ fontSize: 10.5, color: "var(--faint)", lineHeight: 1.5 }}>
            This is what landed on screen. The send didn't go out
            {delivery.reason === "no channel configured" ? " – no channel is wired up yet." : "."}
          </span>
        ) : (
          <span
            className="mono pulse"
            style={{
              fontSize: 9.5,
              fontWeight: 500,
              letterSpacing: "0.18em",
              color: "var(--money)",
            }}
          >
            ● SENDING...

```

```

        </span>
      )
    }
  />
</div>
</div>
</div>

{done && (
  <div
    className="rise closer"
    style={{ alignItems: "center", marginTop: "auto", paddingTop: 4 }}
  >
    <p className="lede" style={{ maxWidth: 640 }}>
      {delivery?.ok
        ? "That is what it feels like. Now here is what it is worth."
        : "That is the sequence. Now here is what it is worth."}
    </p>
    <button className="go" onClick={onNext}>
      <span>What that&rsquo;s worth</span>
      <Arrow />
    </button>
  </div>
)}
</div>
</Shell>
);
}

```

components/Phone.tsx

TypeScript / React · 99 lines

```

import { BoxGlyph, CalendarGlyph, Clock, DollarGlyph, ListGlyph, MailGlyph } from "../Icons";
import { moneyIn } from "@lib/money";
import type { Act, ActKind } from "@lib/types";

/**
 * No fake status bar, no fake keyboard. On a real phone the real ones render
 * on top of this; a painted one reads as a toy.
 */

const LABEL: Record<ActKind, string> = {
  mail: "Mail",
  calendar: "Calendar",
  box: "Orders",
  list: "Your list",
};

function Glyph({ kind }: { kind: ActKind }) {
  if (kind === "calendar") return <CalendarGlyph />;
  if (kind === "box") return <BoxGlyph />;
  if (kind === "list") return <ListGlyph />;
  return <MailGlyph />;
}

function Icon({ kind }: { kind: ActKind | "money" }) {
  return (
    <span className={kind === "money" ? "appIcon moneyIcon" : "appIcon"}>
      {kind === "money" ? <DollarGlyph /> : <Glyph kind={kind} />}
    </span>
  );
}

function ActCard({ act }: { act: Act }) {
  return (
    <div className="notif rise">
      <div className="notifHead">
        <Icon kind={act.kind} />
        <span>{LABEL[act.kind]}</span>
        <span className="notifWhen">now</span>
      </div>
      <span className="notifBody">
        {act.text.charAt(0).toUpperCase() + act.text.slice(1)}.
      </span>
    </div>
  );
}

export interface PhoneState {
  price: number;
  customerName: string;
  purchase: string;
  actNow: Act;
  actSecond: Act;
  followUp: string;
  /** which cards have landed */
  paid: boolean;
  didNow: boolean;
  didSecond: boolean;
  queued: boolean;
  footer: React.ReactNode;
}

export function Phone(s: PhoneState) {
  return (
    <div className="phone">
      {s.paid && (
        <div className="notif money land">
          <div className="notifHead">

```



```

        <Icon kind="money" />
        <span>Payments</span>
        <span className="notifWhen">now</span>
    </div>
    <span className="notifAmount">{moneyIn(s.price)}</span>
    <span className="notifBody">
        {s.customerName} paid you for {s.purchase}.
    </span>
</div>
)}}

{s.didNow && <ActCard act={s.actNow} />}
{s.didSecond && <ActCard act={s.actSecond} />}

{s.queued && (
    <div className="notif waiting rise">
        <div className="notifHead">
            <Clock size={13} color="var(--faint)" />
            <span style={{ color: "var(--faint)" }}>Waiting</span>
            <span className="notifWhen">Wed 9:00 AM</span>
        </div>
        <span className="notifBody" style={{ color: "var(--muted)", fontSize: 12 }}>
            {s.followUp}
        </span>
    </div>
)}}

    <div className="notifFoot">{s.footer}</div>
</div>
);
}

```

components/Proof.tsx

TypeScript / React · 199 lines

```

"use client";

import { Shell } from "./Shell";
import { Arrow } from "./Icons";
import { money, moneyIn } from "@lib/money";
import type { Answers } from "@lib/types";

const DEAD = [
  ["YOU", "spot the payment – if you happen to look"],
  ["YOU", "open ChatGPT and explain the whole thing again"],
  ["AI", "writes a lovely reply. Its job ends here."],
  ["YOU", "copy it, paste it, fix the tone, send it"],
  ["YOU", "write them down somewhere, then try to remember Wednesday"],
] as const;

const LIVE = [
  ["IT", "catches the payment the second it lands"],
  ["IT", "writes them down where they'll be remembered"],
  ["AI", "reads them and decides how they should be treated"],
  ["IT", "delivers, records, and parks Wednesday's next offer"],
  ["IT", "buzzes your phone to say it's already handled"],
] as const;

function Row({ who, text, live }: { who: string; text: string; live: boolean }) {
  const isAi = who === "AI";
  return (
    <div className={`stepRow${live ? " isLive" : isAi ? " isAi" : ""}`}>
      <span className="stepWho">{who}</span>
      <span className="stepText">{text}</span>
    </div>
  );
}

export function Proof({ a, onRestart }: { a: Answers; onRestart: () => void }) {
  const price = money(a.price);

  return (
    <Shell step={5} label="THE DIFFERENCE">
      <div className="stack" style={{ gap: "clamp(24px, 3vw, 34px)" }}>
        <div className="stack" style={{ gap: 16, maxWidth: 830 }}>
          <h1 className="h1">
            Same {price}. <em className="italic">Two worlds.</em>
          </h1>
          <p className="lede">
            Both of these use AI. Only one of them is a system. The difference was never
            the AI – it&rsquo;s whether anything is wired to it on either side.
          </p>
        </div>

        <div className="two">
          <div className="cardDead worldDead">
            <div className="stack" style={{ gap: 8 }}>
              <span className="eyebrow">AI AUTOMATION – WHAT YOU HAVE NOW</span>
              <span
                className="h3"
                style={{ color: "var(--faint)", fontFamily: "var(--serif)" }}
              >
                You, working for the AI
              </span>
            </div>
          </div>
          <div className="stack" style={{ gap: 8 }}>
            {DEAD.map(([who, text]) => (
              <Row key={text} who={who} text={text} live={false} />
            ))}
          </div>
          <div className="stack" style={{ gap: 14, marginTop: "auto" }}>
            <div className="rule" />
          </div>
        </div>
      </div>
    </Shell>
  );
}

```

```

<div className="stats">
  <div className="stat">
    <span className="statNum" style={{ color: "var(--faint)" }}>
      14
    </span>
    <span className="statLabel">CLICKS</span>
  </div>
  <div className="stat">
    <span className="statNum" style={{ color: "var(--faint)" }}>
      4m 20s
    </span>
    <span className="statLabel">OF YOUR LIFE</span>
  </div>
  <div className="stat">
    <span className="statNum" style={{ color: "var(--fainter)" }}>
      none
    </span>
    <span className="statLabel">WHILE ASLEEP</span>
  </div>
</div>
<span style={{ fontSize: 12.5, color: "var(--faint)", lineHeight: 1.55 }}>
  Every payment. Every time. Forever. And the {price} that lands at 11pm sits
  there until you wake up – if the customer waits that long.
</span>
</div>
</div>

<div className="cardMoney worldLive">
  <div className="stack" style={{ gap: 8 }}>
    <span className="eyebrow" style={{ color: "var(--money)" }}>
      AN AI SYSTEM – WHAT YOU JUST WATCHED
    </span>
    <span className="h3">The AI, working for you</span>
  </div>
  <div className="stack" style={{ gap: 8 }}>
    {LIVE.map(([who, text]) => (
      <Row key={text} who={who} text={text} live />
    ))}
  </div>
  <div className="stack" style={{ gap: 14, marginTop: "auto" }}>
    <div className="ruleMoney" />
    <div className="stats">
      <div className="stat">
        <span className="statNum" style={{ color: "var(--money)" }}>
          0
        </span>
        <span className="statLabel" style={{ color: "var(--money)" }}>
          CLICKS
        </span>
      </div>
      <div className="stat">
        <span className="statNum" style={{ color: "var(--money)" }}>
          0.9s
        </span>
        <span className="statLabel" style={{ color: "var(--money)" }}>
          OF YOUR LIFE
        </span>
      </div>
      <div className="stat">
        <span className="statNum" style={{ color: "var(--money)" }}>
          all of it
        </span>
        <span className="statLabel" style={{ color: "var(--money)" }}>
          WHILE ASLEEP
        </span>
      </div>
    </div>
  </div>
  <span style={{ fontSize: 12.5, color: "var(--ink-3)", lineHeight: 1.55 }}>
    Wired once. Runs for every payment after that – at 3am, on Christmas, while
    you're sitting in someone's living room.
  </span>
</div>

```

```

        </span>
      </div>
    </div>
  </div>

  /* the money row – the argument in nine words */
  <div className="card ledger">
    <div className="stack" style={{ gap: 8 }}>
      <span className="eyebrow">AND THE PART THAT MATTERS</span>
      <span style={{ fontSize: 16, fontWeight: 600, lineHeight: 1.35 }}>
        What happens to the money
      </span>
    </div>

    <div className="ledgerCell">
      <span className="ledgerNum">{price}</span>
      <span style={{ fontSize: 12, color: "var(--muted)", lineHeight: 1.45 }}>
        Today. Both worlds collect it. This is the part everyone celebrates.
      </span>
    </div>

    <div className="ledgerCell">
      <span className="ledgerNum dead">${0}</span>
      <span style={{ fontSize: 12, color: "var(--muted)", lineHeight: 1.45 }}>
        Wednesday, with a smart pen. Nobody followed up, because nobody remembered.
      </span>
    </div>

    <div className="ledgerCell money">
      <span className="ledgerNum win">{moneyIn(a.price)}</span>
      <span style={{ fontSize: 12, color: "var(--ink-3)", lineHeight: 1.45 }}>
        Wednesday, with a system. It asked for the next one by itself, while you
        slept.
      </span>
    </div>
  </div>

  <div className="closer" style={{ marginTop: "auto", paddingTop: 12 }}>
    <h2 className="h2" style={{ maxWidth: 820 }}>
      You’ve been working for it.
    <br />
    <em className="italic">Make it work for you</em> – that’s what
    it’s for.
  </h2>
  <div className="stack" style={{ gap: 12, alignItems: "flex-end" }}>
    <button className="go" onClick={onRestart}>
      <span>Wire my business for real</span>
      <Arrow />
    </button>
    <span
      className="mono"
      style={{ fontSize: 9.5, color: "var(--fainter)", letterSpacing: "0.16em" }}
    >
      {a.firstName.toUpperCase()} – WE ALREADY HAVE YOUR NUMBER
    </span>
  </div>
</div>
</div>
</Shell>
);
}

```

components/Rail.tsx

TypeScript / React · 65 lines

```

import { PARTS } from "@lib/anatomy";
import type { PartId } from "@lib/types";

/**
 * The teaching device. Five stations, always five across, at every screen
 * width – collapsing it to two columns on a phone would destroy the one idea
 * the whole demo exists to plant.
 */
export function Rail({
  done,
  now,
  subs,
}: {
  /** parts fully complete */
  done: Set<PartId>;
  /** the part currently running, if any */
  now?: PartId | null;
  /** optional caption under each station, index-aligned with PARTS */
  subs?: (string | null)[];
}) {
  const lastOn = PARTS.reduce(
    (acc, p, i) => (done.has(p.id) || p.id === now ? i : acc),
    -1,
  );

  // The fill has to stop dead on a station node. Node i is centred in grid
  // column i, so its position depends on the column gap as well as the width –
  // the arithmetic lives in CSS (see --rail-col), and all it needs from here
  // is which node to stop at.
  return (
    <div className="rail">
      <div className="railLine" />
      <div
        className="railFill"
        /* the comet head is the leading edge of a pulse in motion – once the
           whole run is complete there is nothing left to lead */
        data-complete={lastOn === PARTS.length - 1 && !now ? "" : undefined}
        style={
          {
            "--i": Math.max(lastOn, 0),
            opacity: lastOn < 0 ? 0 : 1,
          } as React.CSSProperties
        }
      />
      <div className="railGrid">
        {PARTS.map((p, i) => {
          const on = done.has(p.id);
          const running = p.id === now;
          return (
            <div className="station" key={p.id}>
              <div className={`dot${running ? " now pulse" : " on" : ""}`} />
              <div style={{ display: "flex", flexDirection: "column", gap: 3 }}>
                <span className={`stationName${on || running ? " on" : ""}`}>
                  {p.label}
                </span>
                {subs?.[i] && <span className="stationSub">{subs[i]}</span>}
              </div>
            </div>
          );
        })}
      </div>
    </div>
  );
}

```

components/Shell.tsx

TypeScript / React · 45 lines

```
import { WireMark } from "../Icons";

export function Shell({
  step,
  label,
  live,
  children,
}): {
  step: number;
  label: string;
  live?: boolean;
  children: React.ReactNode;
} {
  return (
    <div className="screen">
      <div className="wrap">
        <header className="head">
          <div className="mark">
            <span className="markBadge">
              <WireMark />
            </span>
            <span>THE WIRE</span>
          </div>
          <div className="headRight">
            {live && (
              <span className="livePill">
                <i className="liveDot pulse" />
                LIVE
              </span>
            )}
            { /* five ticks for five screens – the shape of the product, twice */ }
            <div className="ticks" role="presentation" aria-hidden="true">
              {[1, 2, 3, 4, 5].map((n) => (
                <span key={n} className={n <= step ? "tick on" : "tick"} />
              ))}
            </div>
            <span className="stepLabel">{label}</span>
          </div>
        </header>
        <main className="body enter">{children}</main>
      </div>
    </div>
  );
}
```

lib/anatomy.ts

TypeScript · 59 lines

```

import type { PartId } from "./types";

/**
 * The teaching. Five parts, in order, in plain words.
 *
 * This is the one thing a viewer should still be able to recite a week later,
 * so the wording here is the wording everywhere – screens, logs, emails.
 */
export interface Part {
  id: PartId;
  label: string;
  /** what it means, for someone who has never heard the word "webhook" */
  plain: (a: { price: string; sells: string }) => string;
  /** the actual tool doing it, lowercase, unglamorous on purpose */
  tool: string;
}

export const PARTS: Part[] = [
  {
    id: "trigger",
    label: "TRIGGER",
    plain: ({ price }) =>
      `Something real happens. Somebody pays you ${price}. Nobody presses a button – not you, not anyone.`,
    tool: "your payment link",
  },
  {
    id: "capture",
    label: "CAPTURE",
    plain: () =>
      "They get written down somewhere that remembers them. This is the part almost nobody has.",
    tool: "a plain spreadsheet",
  },
  {
    id: "decide",
    label: "DECIDE",
    plain: ({ price }) =>
      `The AI reads them and judges. Paid or just curious? New or returning? Worth ${price} or worth ten times that?`,
    tool: "the ai – this part only",
  },
  {
    id: "act",
    label: "ACT",
    plain: () =>
      "It does the next thing without asking – whatever your business actually owes someone who just paid – and parks a follow-up for Wednesday.",
    tool: "email · your records",
  },
  {
    id: "notify",
    label: "NOTIFY",
    plain: () =>
      "Your phone buzzes with the only sentence that matters. Not a task to do – a result to read.",
    tool: "your phone",
  },
];

export const PART_INDEX: Record<PartId, number> = Object.fromEntries(
  PARTS.map((p, i) => [p.id, i]),
) as Record<PartId, number>;

```

lib/leads.ts

TypeScript · 42 lines

```
import type { Answers } from "./types";

/**
 * The whole point of the demo, commercially.
 *
 * Nobody reaches the green screen without answering all five questions, so a
 * completed run is a lead who has already watched the system work on their own
 * business – with their price attached, which tells you who can afford you
 * before you ever pick up the phone.
 *
 * Set LEADS_WEBHOOK_URL to a Google Apps Script web-app URL that appends a row
 * to a sheet. Anything else that accepts a JSON POST works too.
 */

export interface Lead extends Answers {
  at: string;
  /** where they came from, if the link carried a ?from= */
  source?: string;
  /** did the notification actually land */
  channels?: string[];
}

export async function recordLead(lead: Lead): Promise<void> {
  const url = process.env.LEADS_WEBHOOK_URL;

  // Always log – Vercel keeps these, so a missing webhook never loses a lead.
  console.log("[the-wire] lead", JSON.stringify(lead));

  if (!url) return;

  try {
    await fetch(url, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(lead),
    });
  } catch (e) {
    // Never let the lead write break the demo – the person is mid-experience.
    console.error("[the-wire] lead webhook failed:", e);
  }
}
```


lib/money.ts

TypeScript · 28 lines

```
const WHOLE = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 0,
});

const CENTS = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  minimumFractionDigits: 2,
});

/** $250 – for headlines and body copy */
export const money = (n: number) => WHOLE.format(n);

/** $250.00 – for the notification, where it has to read like a bank */
export const moneyExact = (n: number) => CENTS.format(n);

/** +$250.00 – the hero string. The whole product, in eight characters. */
export const moneyIn = (n: number) => `+${CENTS.format(n)}`;

/** Clamp whatever they typed into something we can put on screen. */
export function parsePrice(raw: string): number | null {
  const n = Number(String(raw).replace(/^[^0-9.]/g, ""));
  if (!Number.isFinite(n) || n <= 0) return null;
  return Math.min(Math.round(n), 1_000_000);
}
```

lib/notify/channels.ts

TypeScript · 206 lines

```

import type { Answers, Personalized } from "../types";
import { moneyIn } from "../money";
import { buildInvite, nextThursday } from "../ics";

export interface Payload {
  answers: Answers;
  copy: Personalized;
}

export interface ChannelResult {
  channel: "sms" | "email";
  ok: boolean;
  /** why it didn't send — shown to nobody, logged for us */
  reason?: string;
}

/**
 * The one line that has to land. Kept under 160 characters so a text is a
 * single segment, and front-loaded with the money because that is the only
 * part read from a lock screen.
 */
export function buzzLine({ answers, copy }: Payload): string {
  return `${moneyIn(answers.price)} — ${copy.customerName} just paid you for ${copy.purchase}. ${cap(copy.actNow.text)},
  ${copy.actSecond.text}, follow-up parked for Wednesday. You did none of it.`;
}

const cap = (s: string) => s.charAt(0).toUpperCase() + s.slice(1);

/* ----- sms */

/**
 * The only channel that reliably buzzes every phone with no permission prompt,
 * no app install and no home-screen trick. Costs about a penny. Worth it: the
 * buzz is the product.
 */
export async function sendSms(p: Payload): Promise<ChannelResult> {
  const sid = process.env.TWILIO_ACCOUNT_SID;
  const token = process.env.TWILIO_AUTH_TOKEN;
  const from = process.env.TWILIO_FROM;

  if (!sid || !token || !from) {
    return { channel: "sms", ok: false, reason: "not configured" };
  }

  const to = normalizeUsMobile(p.answers.mobile);
  if (!to) return { channel: "sms", ok: false, reason: "unusable number" };

  const body = new URLSearchParams({ To: to, From: from, Body: buzzLine(p) });

  try {
    const res = await fetch(
      `https://api.twilio.com/2010-04-01/Accounts/${sid}/Messages.json`,
      {
        method: "POST",
        headers: {
          Authorization: "Basic " + Buffer.from(`${sid}:${token}`).toString("base64"),
          "Content-Type": "application/x-www-form-urlencoded",
        },
        body,
      },
    );
    if (!res.ok) {
      return { channel: "sms", ok: false, reason: `twilio ${res.status}` };
    }
    return { channel: "sms", ok: true };
  } catch (e) {
    return { channel: "sms", ok: false, reason: String(e) };
  }
}

```

```

    }
  }

  /** US mobiles only for now – anything else we simply don't text. */
  export function normalizeUsMobile(raw: string): string | null {
    const digits = String(raw).replace(/\D/g, "");
    if (digits.length === 10) return `+1${digits}`;
    if (digits.length === 11 && digits.startsWith("1")) return `+${digits}`;
    return null;
  }

  /** ----- email */

  /**
   * The free half. Slower and less certain than a text, but it carries the
   * calendar invite as an attachment and it is the durable proof they still have
   * tomorrow morning.
   */
  export async function sendEmail(p: Payload): Promise<ChannelResult> {
    const key = process.env.RESEND_API_KEY;
    const from = process.env.MAIL_FROM;

    if (!key || !from) {
      return { channel: "email", ok: false, reason: "not configured" };
    }

    const { answers, copy } = p;

    // Only attach an invite when this business actually schedules something.
    // A baker does not need a calendar file, and sending one looks careless.
    const scheduled = copy.actNow.kind === "calendar" || copy.actSecond.kind === "calendar";
    const invite = scheduled
      ? buildInvite({
          organizerName: answers.firstName,
          organizerEmail: answers.email,
          customerName: copy.customerName,
          purchase: copy.purchase,
          start: nextThursday(),
          minutes: 60,
        })
      : null;

    try {
      const res = await fetch("https://api.resend.com/emails", {
        method: "POST",
        headers: {
          Authorization: `Bearer ${key}`,
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          from,
          to: [answers.email],
          subject: `${moneyIn(answers.price)} – ${copy.customerName} paid you`,
          text: receiptText(p),
          html: receiptHtml(p),
          attachments: invite
            ? [{ filename: "appointment.ics", content: Buffer.from(invite).toString("base64")} ]
            : undefined,
        }),
      });
      if (!res.ok) {
        return { channel: "email", ok: false, reason: `resend ${res.status}` };
      }
      return { channel: "email", ok: true };
    } catch (e) {
      return { channel: "email", ok: false, reason: String(e) };
    }
  }

  function receiptText({ answers, copy }: Payload): string {

```

```

return [
  `${moneyIn(answers.price)}`,
  "",
  `${copy.customerName} just paid you for ${copy.purchase}.`,
  "",
  "Here is everything that happened in the nine tenths of a second after that,",
  "without you touching anything:",
  "",
  "  TRIGGER    the payment landed",
  "  CAPTURE    they were written down where they'll be remembered",
  "  DECIDE     ${copy.verdict}",
  "  ACT        ${copy.actNow.text}",
  "  ACT        ${copy.actSecond.text}",
  "  NOTIFY     your phone buzzed",
  "",
  `Wednesday's follow-up is already written: ${copy.followUp}`,
  "It will send itself whether or not you ever open this again.",
  "",
  "That is the whole difference. Not a faster task – a business that keeps",
  "earning while you sleep.",
  "",
  "– The Wire",
].join("\n");
}

function receiptHtml({ answers, copy }: Payload): string {
  const row = (label: string, text: string) => `
    <tr>
      <td style="padding:7px 14px 7px 0;font:500 12px/1.4 ui-monospace,Menlo,monospace;color:#0B7A3E;letter-spacing:.06em;white-space:nowrap;vertical-align:top">${label}</td>
      <td style="padding:7px 0;font:400 14px/1.5 -apple-system,Helvetica,sans-serif;color:#37342E">${text}</td>
    </tr>`;

  return `<div style="background:#FBF9F6;padding:32px 20px">
<div style="max-width:560px;margin:0 auto;background:#FFFFFF;border:1px solid #E7E2DA;border-radius:6px;padding:32px">
  <div style="font:500 34px/1 ui-monospace,Menlo,monospace;color:#0B7A3E">${moneyIn(answers.price)}</div>
  <div style="font:400 16px/1.5 -apple-system,Helvetica,sans-serif;color:#37342E;margin-top:10px">
    ${escapeHtml(copy.customerName)} just paid you for ${escapeHtml(copy.purchase)}.
  </div>
  <div style="height:1px;background:#EFEFE4;margin:24px 0"></div>
  <div style="font:400 13px/1.5 -apple-system,Helvetica,sans-serif;color:#6B655D;margin-bottom:12px">
    Everything that happened in the nine tenths of a second after that, without you touching anything:
  </div>
  <table cellpadding="0" cellspacing="0" style="width:100%">
    ${row("TRIGGER", "The payment landed.")}
    ${row("CAPTURE", "They were written down where they&rsquo;ll be remembered.")}
    ${row("DECIDE", escapeHtml(copy.verdict))}
    ${row("ACT", escapeHtml(cap(copy.actNow.text)) + ".")}
    ${row("ACT", escapeHtml(cap(copy.actSecond.text)) + ".")}
    ${row("NOTIFY", "Your phone buzzed.")}
  </table>
  <div style="height:1px;background:#EFEFE4;margin:24px 0"></div>
  <div style="font:400 14px/1.55 -apple-system,Helvetica,sans-serif;color:#37342E">
    Wednesday&rsquo;s follow-up is already written: <em>${escapeHtml(copy.followUp)}</em><br>
    It will send itself whether or not you ever open this again.
  </div>
  <div style="font:400 13px/1.55 -apple-system,Helvetica,sans-serif;color:#8C857C;margin-top:20px">
    That is the whole difference. Not a faster task &mdash; a business that keeps earning while you sleep.
  </div>
</div>`;
}

function escapeHtml(s: string): string {
  return s
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;");
}

```

lib/notify/ics.ts

TypeScript · 80 lines

```

/**
 * A calendar invite as a file attachment.
 *
 * Deliberately not the Google Calendar API: that needs write-access OAuth, and
 * Google's consent screen is the single scariest thing we could put in front of
 * a non-technical person. An .ics attachment costs zero permissions and every
 * phone offers to add it with one tap.
 */

interface InviteInput {
  organizerName: string;
  organizerEmail: string;
  customerName: string;
  purchase: string;
  /** when the imaginary appointment sits */
  start: Date;
  minutes: number;
}

const stamp = (d: Date) => d.toISOString().replace(/[-:]/g, "").replace(/\\.d{3}/, "");

/** Fold long lines at 75 octets, per RFC 5545 – some clients drop the event otherwise. */
function fold(line: string): string {
  if (line.length <= 75) return line;
  const parts = [line.slice(0, 75)];
  let rest = line.slice(75);
  while (rest.length > 74) {
    parts.push(" " + rest.slice(0, 74));
    rest = rest.slice(74);
  }
  if (rest) parts.push(" " + rest);
  return parts.join("\r\n");
}

const esc = (s: string) => s.replace(/([,;\\])/g, "\\$1").replace(/\n/g, "\\n");

export function buildInvite(input: InviteInput): string {
  const end = new Date(input.start.getTime() + input.minutes * 60_000);
  const uid = `${crypto.randomUUID()}@the-wire`;

  const lines = [
    "BEGIN:VCALENDAR",
    "VERSION:2.0",
    "PRODID:-//The Wire//EN",
    "CALSCALE:GREGORIAN",
    "METHOD:PUBLISH",
    "BEGIN:VEVENT",
    `UID:${uid}`,
    `DTSTAMP:${stamp(new Date())}`,
    `DTSTART:${stamp(input.start)}`,
    `DTEND:${stamp(end)}`,
    fold(`SUMMARY:${esc(`${input.customerName} – ${input.purchase}`)}`),
    fold(
      `DESCRIPTION:${esc(
        `${input.customerName} paid and booked this herself. Nobody typed it in.\n\nThis is what a wired system does with a payment:
        it captures the customer, decides how to treat her, acts, and tells you afterwards.`
      )}`,
    ),
    fold(`ORGANIZER;CN=${esc(input.organizerName)}:mailto:${input.organizerEmail}`),
    "STATUS:CONFIRMED",
    "BEGIN:VALARM",
    "TRIGGER:-PT15M",
    "ACTION:DISPLAY",
    "DESCRIPTION:Reminder",
    "END:VALARM",
    "END:VEVENT",
    "END:VCALENDAR",
  ];

```

```
    ];  
  
    return lines.join("\r\n");  
}  
  
/** Thursday of this week at 2:15pm local-ish, or next Thursday if that has passed. */  
export function nextThursday(from = new Date()): Date {  
    const d = new Date(from);  
    d.setHours(14, 15, 0, 0);  
    const delta = (4 - d.getDay() + 7) % 7 || 7;  
    d.setDate(d.getDate() + delta);  
    return d;  
}
```

lib/notify/index.ts

TypeScript · 36 lines

```
import { sendEmail, sendSms, type ChannelResult, type Payload } from "./channels";

export type { Payload, ChannelResult };
export { buzzLine } from "./channels";

/**
 * Fire every configured channel at once and report which ones landed.
 *
 * Called BEFORE the on-screen animation starts, never after: the screen is
 * paced to about seven seconds, so the phone can only ever be early. Early
 * reads as magic. Late reads as broken.
 */
export async function dispatch(p: Payload): Promise<ChannelResult[]> {
  const results = await Promise.all([sendSms(p), sendEmail(p)]);

  for (const r of results) {
    if (!r.ok && r.reason !== "not configured") {
      console.error(`[the-wire] ${r.channel} failed: ${r.reason}`);
    }
  }

  return results;
}

/** True when at least one channel is wired up — the page adapts its promise. */
export function configuredChannels(): Array<"sms" | "email"> {
  const out: Array<"sms" | "email"> = [];
  if (process.env.TWILIO_ACCOUNT_SID && process.env.TWILIO_AUTH_TOKEN && process.env.TWILIO_FROM) {
    out.push("sms");
  }
  if (process.env.RESEND_API_KEY && process.env.MAIL_FROM) {
    out.push("email");
  }
  return out;
}
```

lib/personalize.ts

TypeScript · 119 lines

```

import Anthropic from "@anthropic-ai/sdk";
import { z } from "zod";
import { zodOutputFormat } from "@anthropic-ai/sdk/helpers/zod";
import type { Answers, Personalized } from "./types";
import { money } from "./money";

/**
 * The demo must never wait on this.
 *
 * It runs the moment they submit the form, while they are still reading the
 * diagnosis and the anatomy – roughly forty seconds of runway – and the result
 * is held on the client until they pull the trigger. If the model is slow or
 * unreachable, `fallback()` is already good enough to ship, so the magic moment
 * is never gambled on a network round-trip.
 *
 * The whole point of generating this is that the demo has to fit anyone with
 * something to sell. A baker does not book appointments; a consultant does not
 * ship boxes. So the two actions are written for THEIR business rather than
 * assumed.
 */

const ActSchema = z.object({
  text: z
    .string()
    .describe("One clipped log line, lowercase, under 12 words. No pronouns like 'I'."),
  kind: z
    .enum(["mail", "calendar", "box", "list"])
    .describe(
      "mail = something was sent. calendar = something was scheduled or held. box = something physical or a digital file was
dispatched or unlocked. list = a record was updated.",
    ),
});

const CopySchema = z.object({
  customerName: z
    .string()
    .describe("A plausible full name for their customer. Ordinary, not exotic."),
  purchase: z
    .string()
    .describe(
      "What the customer bought, in the seller's own words, lowercase, no quotes, under 8 words.",
    ),
  verdict: z
    .string()
    .describe(
      "What the AI concluded about this customer, as one clipped log line under 12 words.",
    ),
  actNow: ActSchema.describe(
    "The first thing the system does FOR THE CUSTOMER, immediately. Whatever this business would actually owe someone the second
they paid.",
  ),
  actSecond: ActSchema.describe(
    "The second thing, done for the OWNER – their records, their schedule, their stock. Must be a different kind from actNow.",
  ),
  followUp: z
    .string()
    .describe(
      "The follow-up the system sends three days later. One sentence, under 20 words, concrete to this business, and aimed at a
second sale.",
    ),
});

const SYSTEM = `You write the copy for a live demo that shows business owners what a wired AI system does the moment somebody pays
them.`

```

The reader is not technical, and they could be selling anything: coaching, cakes, cleaning, houses, courses, jewellery, bookkeeping, dog training, software. Never assume they take appointments, never assume they ship anything, never assume they have staff. Read

what they actually sell and write only what would genuinely happen in that business.

Write the way their smartest customer would talk. Plain words, specific nouns, no marketing voice, no exclamation marks, no em-dashes, no words like "seamless", "leverage", "unlock" or "journey". Refer to the customer by name or as "them" – never guess their gender.`;

```
function prompt(a: Answers): string {
  return `They sell: ${a.sells}
  They charge: ${money(a.price)}`
}
```

Invent the single customer who just paid them. Then write the two things a real system would do in the first second, and the follow-up it would send three days later.

The follow-up must be a plausible next step for someone who just bought this – not a satisfaction survey, not "just checking in".`;

```
/** Good enough to ship on its own, and true for any business on earth. */
export function fallback(a: Answers): Personalized {
  return {
    customerName: "Maria Ruiz",
    purchase: a.sells.trim().toLowerCase().replace(/\.$/ , ""),
    verdict: "new customer, paid in full, no history – treat as high value",
    actNow: {
      text: "receipt and welcome note sent from your own address",
      kind: "mail",
    },
    actSecond: {
      text: "added to your customer list with everything they bought",
      kind: "list",
    },
    followUp: `Ask how it went and offer them the next thing you sell.`,
    live: false,
  };
}

export async function personalize(a: Answers): Promise<Personalized> {
  if (!process.env.ANTHROPIC_API_KEY) return fallback(a);

  try {
    const client = new Anthropic();

    const response = await client.messages.parse({
      model: "claude-opus-5",
      max_tokens: 1024,
      system: SYSTEM,
      // Short, mechanical generation on a stopwatch – depth buys nothing here,
      // and every saved second is runway we don't have to hide.
      output_config: { effort: "low", format: zodOutputFormat(CopySchema) },
      messages: [{ role: "user", content: prompt(a) }],
    });

    const out = response.parsed_output;
    if (!out) return fallback(a);

    return { ...out, live: true };
  } catch (e) {
    console.error("[the-wire] personalize failed, using fallback:", e);
    return fallback(a);
  }
}
```

lib/types.ts

TypeScript · 71 lines

```

/** The five answers. This is the entire onboarding, and the entire lead record. */
export interface Answers {
  firstName: string;
  email: string;
  mobile: string;
  sells: string;
  /** whole dollars – the number that turns the whole demo green */
  price: number;
}

/** The five parts every AI system is made of. Order is load-bearing. */
export type PartId = "trigger" | "capture" | "decide" | "act" | "notify";

export interface Step {
  /** monotonic index so the client can dedupe a reconnected stream */
  n: number;
  part: PartId;
  /** ms since the trigger fired, as the server measured it */
  at: number;
  text: string;
  /** renders green and larger – reserved for money actually moving */
  money?: string;
  /** this step is parked for the future, not done now */
  scheduled?: string;
}

export type FireEvent =
  | { type: "step"; step: Step }
  | { type: "delivered"; channels: string[]; at: number }
  | { type: "failed"; reason: string; at: number }
  | { type: "done"; total: number; elapsed: number };

/** Copy generated from their answers, cached before they ever pull the trigger. */
export interface Personalized {
  /** the imaginary customer who pays them */
  customerName: string;
  /** what the customer bought, in their words */
  purchase: string;
  /** what the AI "decided" about this customer */
  verdict: string;
  /** what the system does for the customer immediately */
  actNow: Act;
  /** what it does for the owner's own records or schedule */
  actSecond: Act;
  /** the follow-up the system parks for three days out */
  followUp: string;
  /** true when this came from the model rather than the fallback */
  live: boolean;
}

/**
 * What kind of thing an action produced – this picks the icon on the phone.
 * The demo has to fit a coach, a baker and a web developer equally well, so
 * the actions are written per business rather than assumed to be appointments.
 */
export type ActKind = "mail" | "calendar" | "box" | "list";

export interface Act {
  text: string;
  kind: ActKind;
}

export interface Run {
  id: string;
  answers: Answers;
  createdAt: number;
  /** resolves while they read screens 2 and 3, so the trigger never waits */

```

```
    copy: Promise<Personalized>;  
    firedAt?: number;  
  }
```

lib/validate.ts

TypeScript · 59 lines

```

import type { Act, ActKind, Answers, Personalized } from "./types";
import { parsePrice } from "./money";

const KINDS: ActKind[] = ["mail", "calendar", "box", "list"];

const clip = (s: unknown, max: number) => String(s ?? "").trim().slice(0, max);

/** Loose on purpose – this is a demo, not a checkout. We only reject unusable. */
export function readAnswers(raw: unknown): Answers | null {
  if (!raw || typeof raw !== "object") return null;
  const r = raw as Record<string, unknown>;

  const firstName = clip(r.firstName, 40);
  const email = clip(r.email, 120);
  const mobile = clip(r.mobile, 30);
  const sells = clip(r.sells, 120);
  const price = parsePrice(String(r.price ?? ""));

  if (!firstName || !sells || price === null) return null;
  if (!email.includes("@") || email.startsWith("@") || email.endsWith("@")) return null;

  return { firstName, email, mobile, sells, price };
}

function readAct(raw: unknown, fallbackKind: ActKind): Act | null {
  if (!raw || typeof raw !== "object") return null;
  const r = raw as Record<string, unknown>;
  const text = clip(r.text, 160);
  if (!text) return null;
  const kind = KINDS.includes(r.kind as ActKind) ? (r.kind as ActKind) : fallbackKind;
  return { text, kind };
}

export function readCopy(raw: unknown): Personalized | null {
  if (!raw || typeof raw !== "object") return null;
  const r = raw as Record<string, unknown>;

  const customerName = clip(r.customerName, 60);
  const purchase = clip(r.purchase, 120);
  const verdict = clip(r.verdict, 160);
  const followUp = clip(r.followUp, 200);
  const actNow = readAct(r.actNow, "mail");
  const actSecond = readAct(r.actSecond, "list");

  if (!customerName || !purchase || !verdict || !followUp || !actNow || !actSecond) {
    return null;
  }

  return {
    customerName,
    purchase,
    verdict,
    actNow,
    actSecond,
    followUp,
    live: Boolean(r.live),
  };
}

```

next.config.ts

TypeScript · 8 lines

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
};

export default nextConfig;
```

package.json

JSON · 24 lines

```
{
  "name": "the-wire",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "@anthropic-ai/sdk": "^0.122.0",
    "next": "16.3.3",
    "react": "19.2.8",
    "react-dom": "19.2.8",
    "zod": "^4.5.2"
  },
  "devDependencies": {
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "typescript": "^5"
  }
}
```

tsconfig.json

JSON · 35 lines

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```